

thesis_project

March 13, 2025



1 Predicting Housing Market Trends Using Data Science

1.0.1 Potential Insights:

The housing market is influenced by a complex mix of economic conditions, supply and demand, and regional variations. Understanding these trends allows buyers, sellers, and investors to make informed decisions.

This dataset provides insights into housing market trends across different regions. I chose this dataset because:

- Housing market data reflects economic patterns, consumer behavior, and financial stability.
- By analyzing price fluctuations, sales trends, and inventory changes, I aim to identify key indicators that predict market shifts.
- The dataset includes multiple region types, allowing for comparisons between national trends and city-specific behaviors.
- By leveraging machine learning techniques, I will analyze leading indicators of price changes and market cycles.

1.0.2 Project Scope

Objectives:

- Identify key trends and patterns in the housing market.
- Predict market fluctuations using data-driven insights.
- Provide actionable insights into real estate trends.

Deliverables:

- Charts, graphs, statistical reports, predictive models.

- A final research paper detailing insights and methodologies.

Milestones:

- Week 1: Data collection and cleaning.
- Weeks 2-3: Exploratory Data Analysis (EDA).
- Weeks 4-6: Model selection and training.
- Weeks 7-8: Model validation and optimization.
- Weeks 9-10: Final report and presentation.

Tasks:

- Data acquisition and preprocessing
- Exploratory data analysis (EDA)
- Feature engineering and selection
- Model training and validation
- Performance evaluation and optimization
- Presentation of findings

Resources: Python, Pandas, NumPy, Scikit-Learn, Matplotlib, Seaborn, Jupyter Notebook

1.0.3 Research Plan

Techniques and Methods:

- Time-series analysis to identify long-term trends.
- Regression models to analyze price determinants.
- Machine learning techniques to predict price changes.
- Cross-validation methods to ensure robust model performance.

Hypothesis & Research Questions**- Hypothesis Statement:**

Housing market trends can be predicted by analyzing historical price fluctuations, inventory changes, and sales patterns. By applying machine learning techniques, we can identify early indicators of market shifts, allowing for better-informed real estate investment decisions.

Research Questions

1. What factors most influence median sale prices?
2. How do changes in inventory levels affect future prices?
3. Are seasonal trends significant in price fluctuations?
4. Can we predict price changes using historical data?

2 Imports

```
[1]: # my imports
import wrangle as w
import explore as e
import modeling as m

# ignore warnings
import warnings
warnings.filterwarnings("ignore")
```

imports loaded successfully, awaiting commands...

2.1 # Wrangle

- Raw Data acquired from Redfin
- 1099 rows x 20 columns before cleaning
- Data acquired from Google BigQuery after cleaning
- 1099 rows x 20 columns after cleaning

2.2 ## Prepare

- Clean the data
 - Rename columns
 - Remove nulls
 - Remove extra characters
 - Remove extra spaces
 - Fixed data type

```
[2]: redfin_df = w.check_file_exists_gbq("data.csv", "service-account-key.json",  
                                         "iu-thesis-project."  
                                         ↪Redfin_Monthly_Housing_Market_Data.Redfin")
```

CSV file 'data.csv' found locally. Checking data quality...
Detected encoding: ascii
Data appears to be clean.
Data is already clean. No need to clean or upload.

2.3 ### A Brief Look At The Data

```
[3]: redfin_df.head()
```

```
[3]:
```

	region	month_of_period_end	median_sale_price	\
0	National	2022-07-01 00:00:00+00:00	413000	
1	National	2022-08-01 00:00:00+00:00	407000	
2	National	2022-06-01 00:00:00+00:00	429000	
3	National	2021-08-01 00:00:00+00:00	381000	
4	National	2021-09-01 00:00:00+00:00	377000	

	median_sale_price_mom	median_sale_price_yoy	homes_sold	homes_sold_mom	\
0	-3.6	7.4	530476	-14.4	
1	-1.5	6.9	559150	5.4	
2	-0.7	10.8	619857	4.1	
3	-1.1	16.2	672612	-1.0	
4	-1.0	13.9	638337	-5.1	

	homes_sold_yoy	new_listings	new_listings_mom	new_listings_yoy	\
0	-21.9	705215	-14.3	-11.9	
1	-16.9	622537	-11.7	-13.7	
2	-15.0	822855	6.4	1.0	

3	0.1	721768	-9.8	0.0
4	-4.7	675008	-6.5	-4.5

	inventory	inventory_mom	inventory_yoy	days_on_market \
0	1240506	10.2	27.1	21
1	1225537	-1.2	26.7	27
2	1125321	20.7	28.1	18
3	967307	-0.9	-18.2	17
4	969546	0.2	-16.1	19

	days_on_market_mom	days_on_market_yoy	average_sale_to_list \
0	3	6	101.0
1	5	10	99.9
2	1	3	102.2
3	2	-15	101.6
4	2	-11	101.0

	average_sale_to_list_mom	average_sale_to_list_yoy
0	-1.2	-1.2
1	-1.1	-1.7
2	-0.8	-0.3
3	-0.6	2.4
4	-0.6	1.7

2.4 ### A Summary Of The Data

```
[4]: e.data_summary(redfin_df)
```

Data shape: (1099, 20)

```
[4]:
```

	data type	#missing	%missing	#unique \
region	object	0	0.0	7
month_of_period_end	datetime64[ns, UTC]	0	0.0	157
median_sale_price	int64	0	0.0	416
median_sale_price_mom	float64	0	0.0	186
median_sale_price_yoy	float64	0	0.0	251
homes_sold	int64	0	0.0	1036
homes_sold_mom	float64	0	0.0	556
homes_sold_yoy	float64	0	0.0	490
new_listings	int64	0	0.0	1062
new_listings_mom	float64	0	0.0	634
new_listings_yoy	float64	0	0.0	462
inventory	int64	0	0.0	1084
inventory_mom	float64	0	0.0	367
inventory_yoy	float64	0	0.0	558
days_on_market	int64	0	0.0	100
days_on_market_mom	int64	0	0.0	45

days_on_market_yoy	int64	0	0.0	86
average_sale_to_list	float64	0	0.0	130
average_sale_to_list_mom	float64	0	0.0	43
average_sale_to_list_yoy	float64	0	0.0	99

	count	mean	\
region	1099	NaN	
month_of_period_end	1099	2018-07-01 19:52:21.401273856+00:00	
median_sale_price	1099.0	407515.013649	
median_sale_price_mom	1099.0	0.610464	
median_sale_price_yoy	1099.0	6.849045	
homes_sold	1099.0	73960.789809	
homes_sold_mom	1099.0	1.37152	
homes_sold_yoy	1099.0	1.94222	
new_listings	1099.0	89187.496815	
new_listings_mom	1099.0	3.776797	
new_listings_yoy	1099.0	0.10919	
inventory	1099.0	224060.347589	
inventory_mom	1099.0	-0.101001	
inventory_yoy	1099.0	-4.468062	
days_on_market	1099.0	38.658781	
days_on_market_mom	1099.0	-0.213831	
days_on_market_yoy	1099.0	-3.524113	
average_sale_to_list	1099.0	99.071338	
average_sale_to_list_mom	1099.0	0.020291	
average_sale_to_list_yoy	1099.0	0.301911	

	std	min	\
region	NaN	NaN	
month_of_period_end	NaN	2012-01-01 00:00:00+00:00	
median_sale_price	189404.64317	134000.0	
median_sale_price_mom	3.657105	-12.8	
median_sale_price_yoy	5.72749	-10.7	
homes_sold	174049.251215	974.0	
homes_sold_mom	18.047561	-40.6	
homes_sold_yoy	17.349035	-56.9	
new_listings	210707.155307	1010.0	
new_listings_mom	29.206833	-59.5	
new_listings_yoy	17.261469	-70.1	
inventory	541161.876914	1057.0	
inventory_mom	9.783868	-39.3	
inventory_yoy	23.895725	-60.5	
days_on_market	21.157081	5.0	
days_on_market_mom	6.726423	-49.0	
days_on_market_yoy	12.70981	-69.0	
average_sale_to_list	2.213431	92.0	
average_sale_to_list_mom	0.510114	-4.1	

average_sale_to_list_yoy	1.418228	-10.6
--------------------------	----------	-------

		25% \
region		NaN
month_of_period_end	2015-04-01 00:00:00+00:00	
median_sale_price		256500.0
median_sale_price_mom		-1.8
median_sale_price_yoy		3.35
homes_sold		3283.0
homes_sold_mom		-10.7
homes_sold_yoy		-6.5
new_listings		3613.0
new_listings_mom		-13.0
new_listings_yoy		-6.8
inventory		7138.0
inventory_mom		-4.45
inventory_yoy		-17.4
days_on_market		23.0
days_on_market_mom		-2.0
days_on_market_yoy		-8.0
average_sale_to_list		97.7
average_sale_to_list_mom		-0.2
average_sale_to_list_yoy		-0.1

		50% \
region		NaN
month_of_period_end	2018-07-01 00:00:00+00:00	
median_sale_price		371000.0
median_sale_price_mom		0.2
median_sale_price_yoy		6.5
homes_sold		5108.0
homes_sold_mom		0.2
homes_sold_yoy		1.8
new_listings		6070.0
new_listings_mom		-2.5
new_listings_yoy		0.2
inventory		12397.0
inventory_mom		0.4
inventory_yoy		-7.3
days_on_market		36.0
days_on_market_mom		1.0
days_on_market_yoy		-2.0
average_sale_to_list		99.1
average_sale_to_list_mom		-0.1
average_sale_to_list_yoy		0.3

75%	max
-----	-----

region	NaN	NaN
month_of_period_end	2021-10-01 00:00:00+00:00	2025-01-01 00:00:00+00:00
median_sale_price	525000.0	925000.0
median_sale_price_mom	2.8	16.4
median_sale_price_yoy	9.7	31.8
homes_sold	7373.0	729414.0
homes_sold_mom	11.55	63.4
homes_sold_yoy	10.7	127.6
new_listings	9152.0	844234.0
new_listings_mom	14.05	150.3
new_listings_yoy	6.75	257.9
inventory	22078.0	2181834.0
inventory_mom	4.55	49.4
inventory_yoy	4.9	230.0
days_on_market	49.0	145.0
days_on_market_mom	4.0	19.0
days_on_market_yoy	3.0	43.0
average_sale_to_list	100.2	110.7
average_sale_to_list_mom	0.3	3.6
average_sale_to_list_yoy	0.9	7.2

```
[5]: redfin_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1099 entries, 0 to 1098
Data columns (total 20 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   region                                1099 non-null   object
1   month_of_period_end                   1099 non-null   datetime64[ns, UTC]
2   median_sale_price                     1099 non-null   int64
3   median_sale_price_mom                 1099 non-null   float64
4   median_sale_price_yoy                 1099 non-null   float64
5   homes_sold                           1099 non-null   int64
6   homes_sold_mom                       1099 non-null   float64
7   homes_sold_yoy                       1099 non-null   float64
8   new_listings                         1099 non-null   int64
9   new_listings_mom                     1099 non-null   float64
10  new_listings_yoy                     1099 non-null   float64
11  inventory                             1099 non-null   int64
12  inventory_mom                         1099 non-null   float64
13  inventory_yoy                         1099 non-null   float64
14  days_on_market                       1099 non-null   int64
15  days_on_market_mom                   1099 non-null   int64
16  days_on_market_yoy                   1099 non-null   int64
17  average_sale_to_list                 1099 non-null   float64
18  average_sale_to_list_mom             1099 non-null   float64
19  average_sale_to_list_yoy             1099 non-null   float64
```

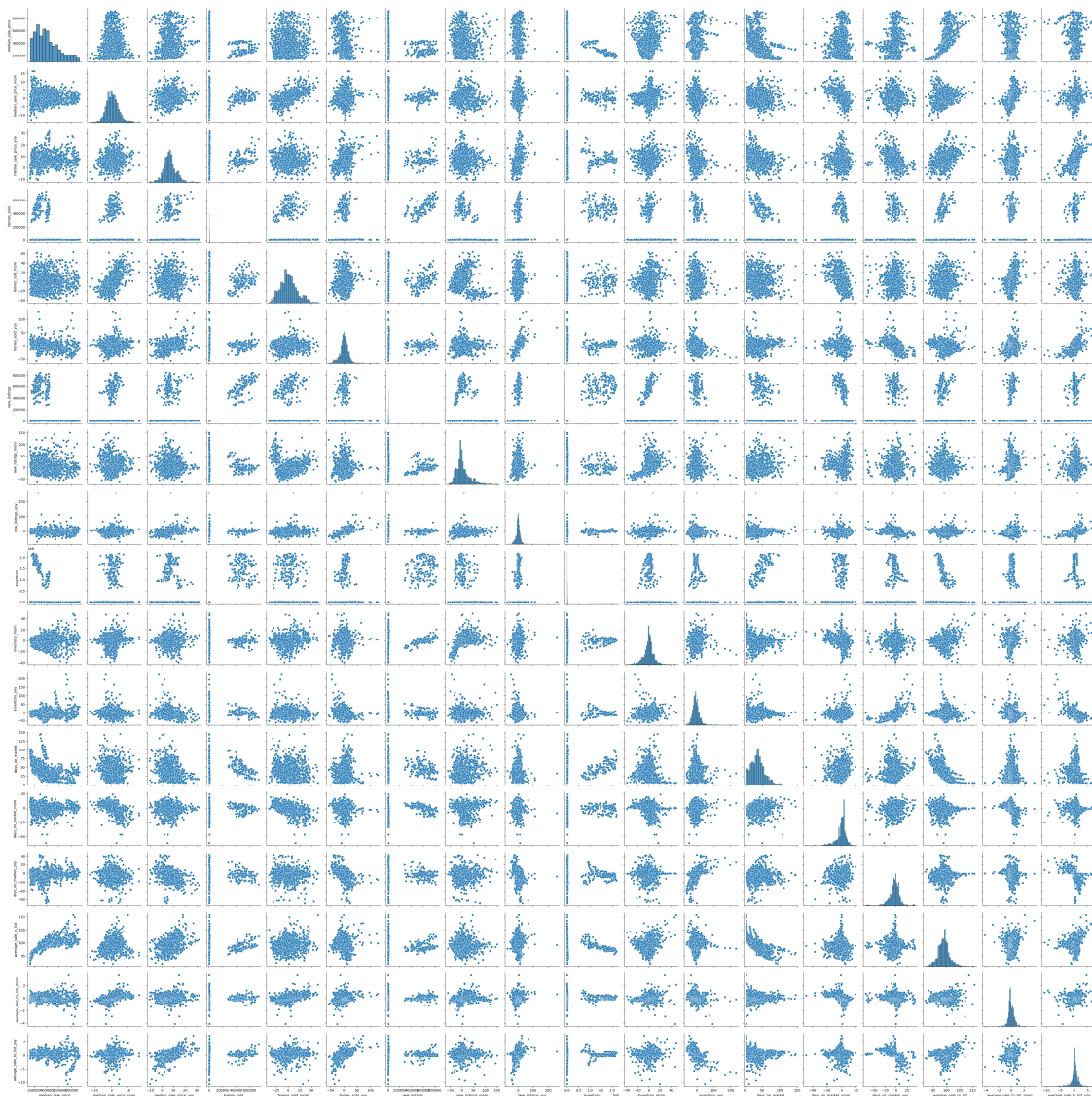
```
dtypes: datetime64[ns, UTC](1), float64(11), int64(7), object(1)
memory usage: 171.8+ KB
```

3 Exploratory Data Analysis (EDA)

```
[6]: # visualized your data
import matplotlib.pyplot as plt
import seaborn as sns

# to view a bit closer the pairplot double click on the plot
sns.pairplot(redfin_df)
```

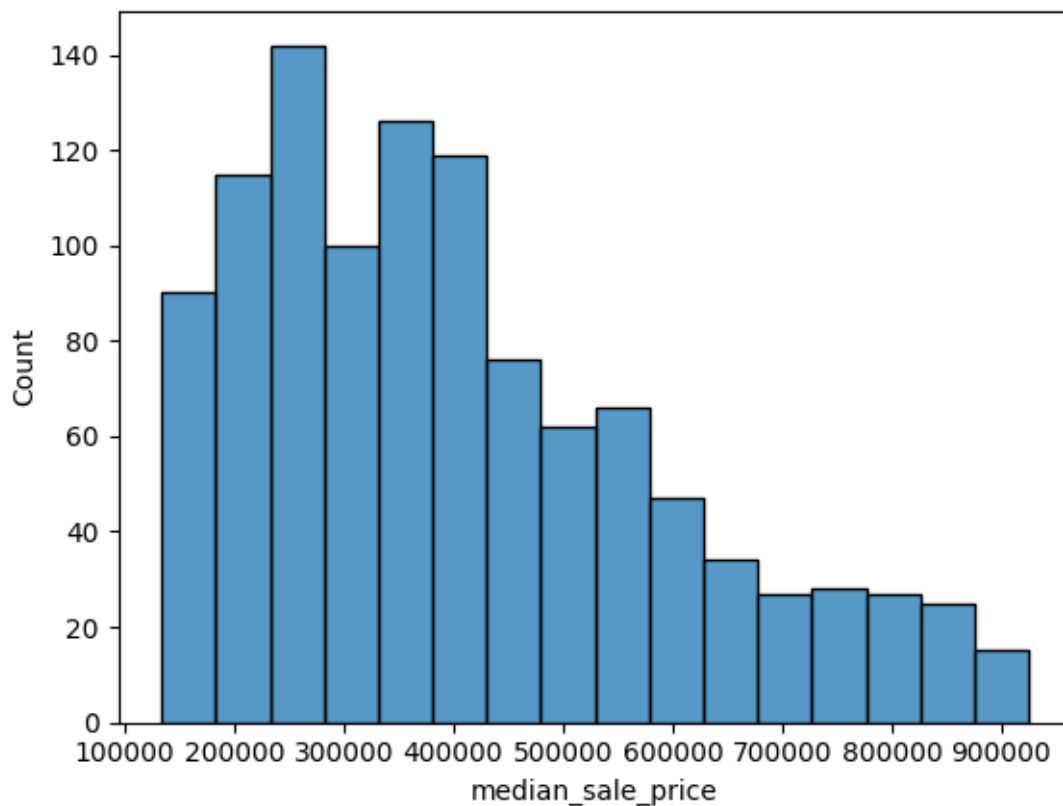
```
[6]: <seaborn.axisgrid.PairGrid at 0x176d09220>
```



What is the distribution of the Target?

- Note: this question is more of a look at the data, not a stats test question.

```
[7]: # get a histplot of quality
sns.histplot(redfin_df.median_sale_price)
plt.show()
```



Target Justification: median_sale_price

1. Core Predictive Goal
 - Both the Project Proposal (focused on housing market trend prediction) and Literature Review (highlighting house-price forecasting as critical) point to home prices as the principal outcome of interest.
 - As emphasized in Assignment 2, “Accurately predicting real estate house prices is critical for financial institutions, investors, and policymakers.”
2. Data Alignment

- The Redfin dataset explicitly provides median_sale_price (along with month-over-month and year-over-year variations).
 - In real estate forecasting literature, the median sale price is a standard measure, serving as a robust, central indicator of market conditions.
3. Common Industry Metric
- Forecasting median sale price is standard practice in both industry and academic settings. It allows analysis of market health and trend over time, region by region.

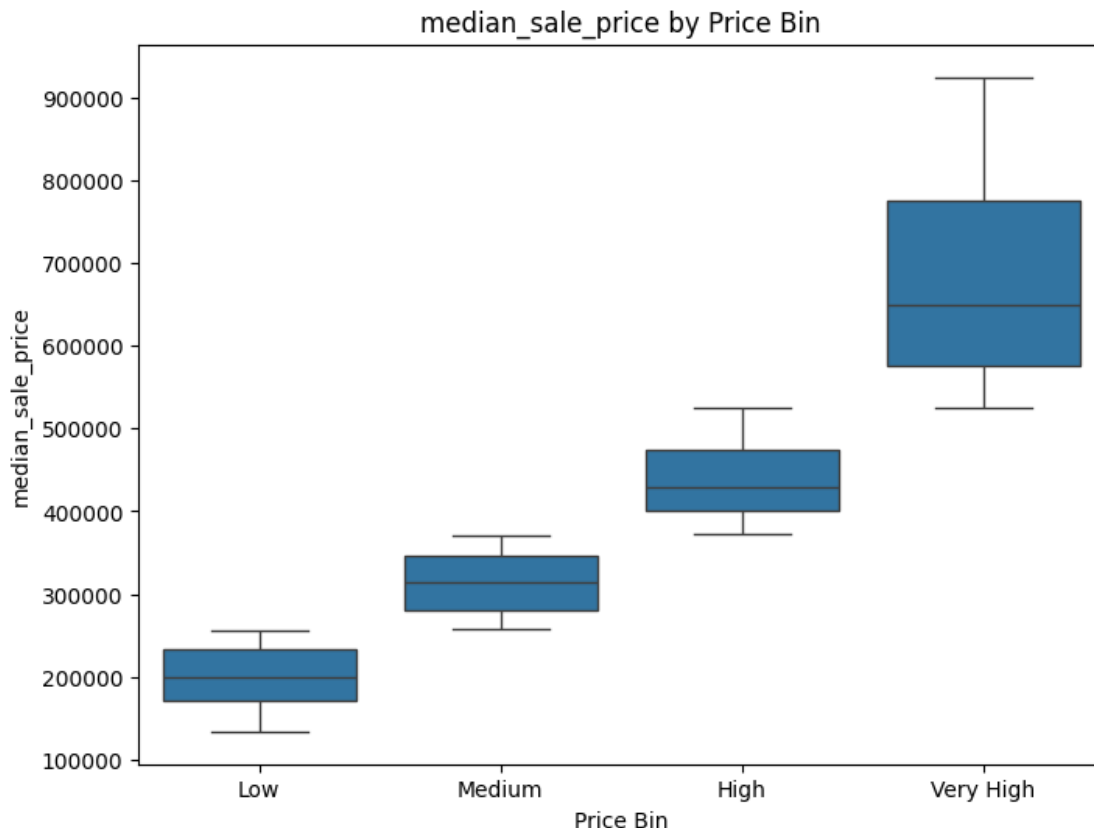
Histogram Insights

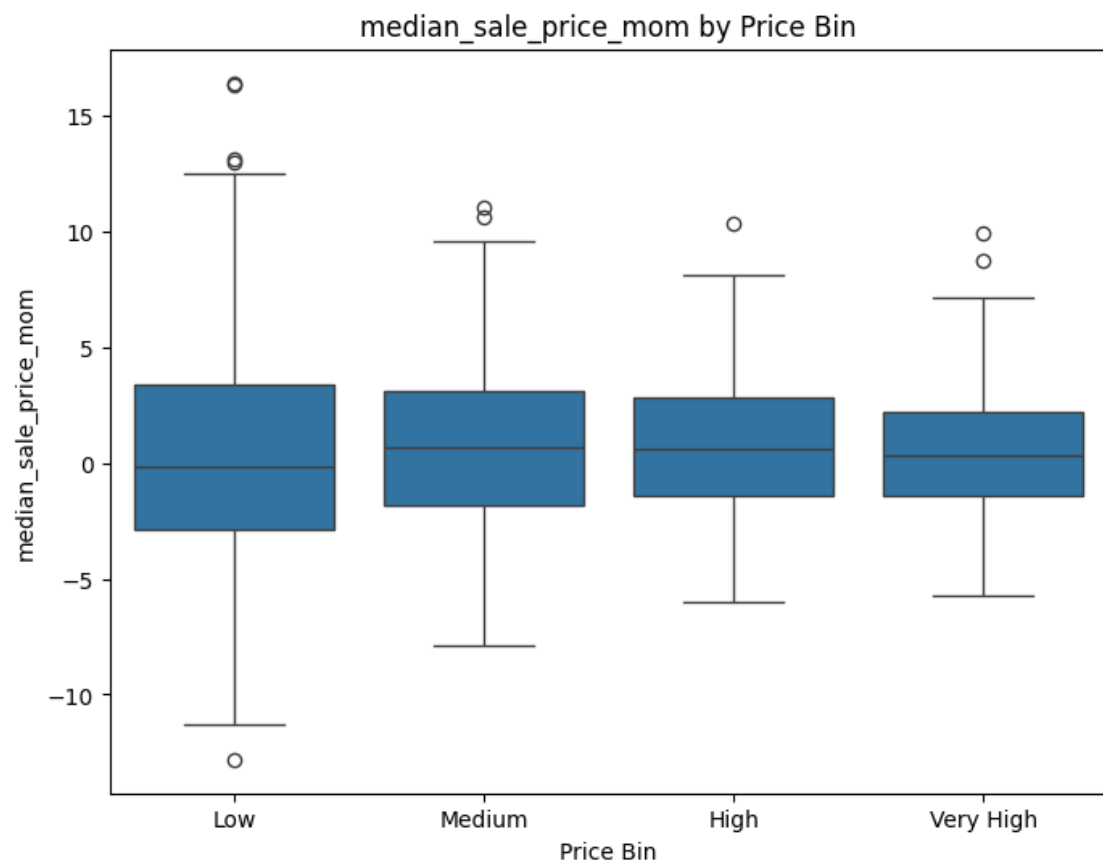
Looking at the histogram of median_sale_price:

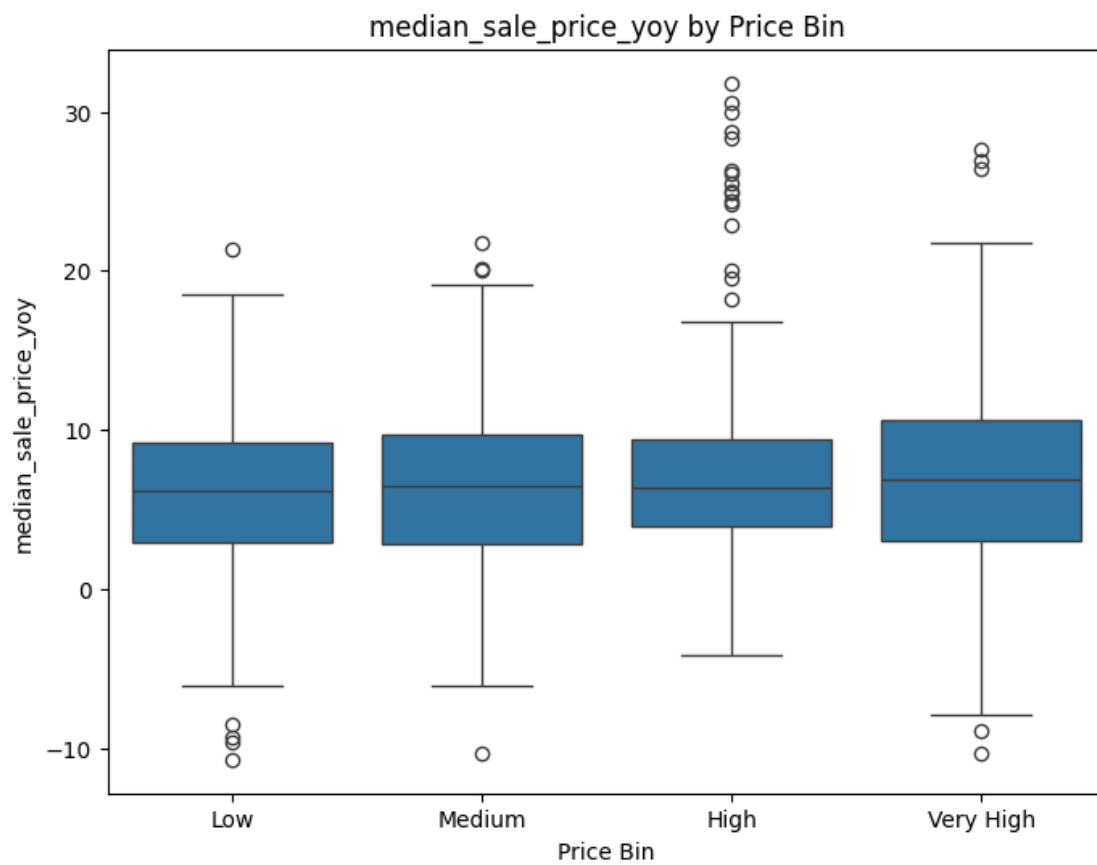
- The distribution peaks roughly in the \ \$200,000 - \ \$300,000 range, indicating that many observations cluster around these mid-range house prices.
- The tail extends toward higher price points (up to \$900,000), reflecting that while most houses fall in the lower-to-mid range, there are notable but fewer properties in higher price brackets.
- The shape suggests a right-skew (common in housing data), where a larger number of transactions occur at relatively moderate price levels, and fewer yet higher-priced properties cause the tail to stretch to the right.

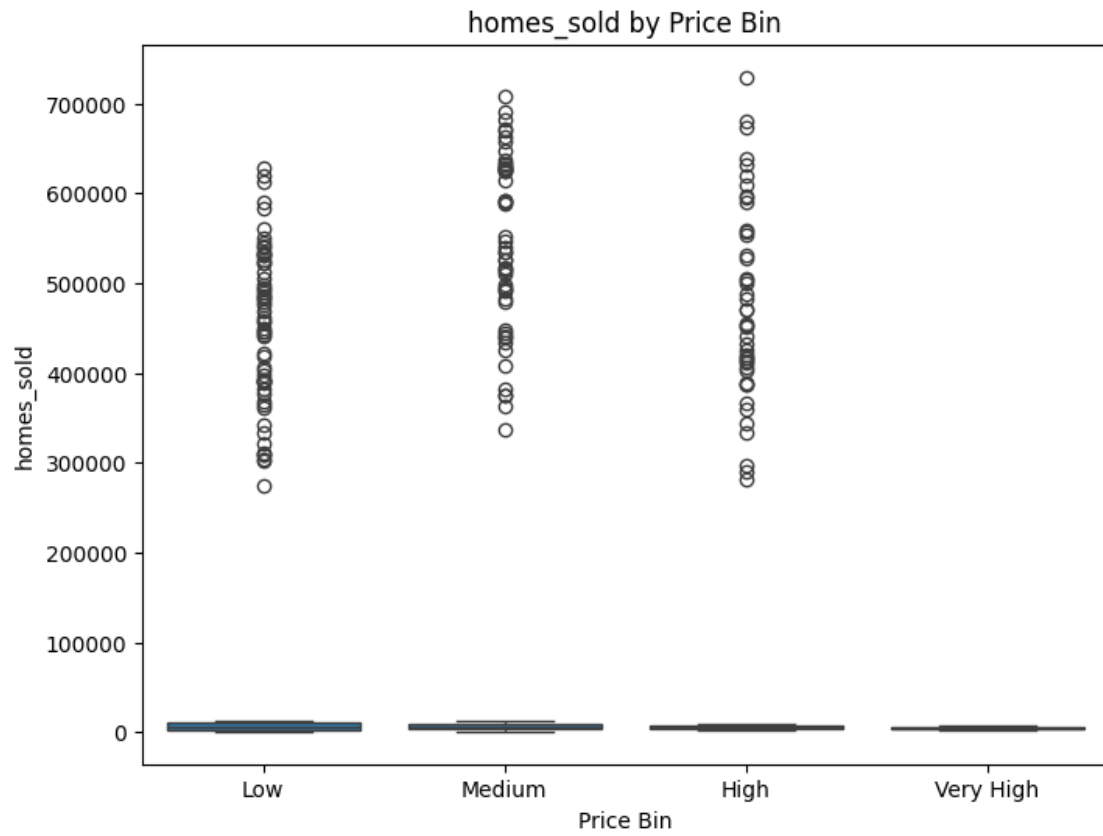
```
[8]: # Identifying numerical and categorical columns
numerical_cols = redfin_df.select_dtypes(include=['int64', 'float64']).columns
categorical_cols = redfin_df.select_dtypes(include=['object', 'bool']).columns
```

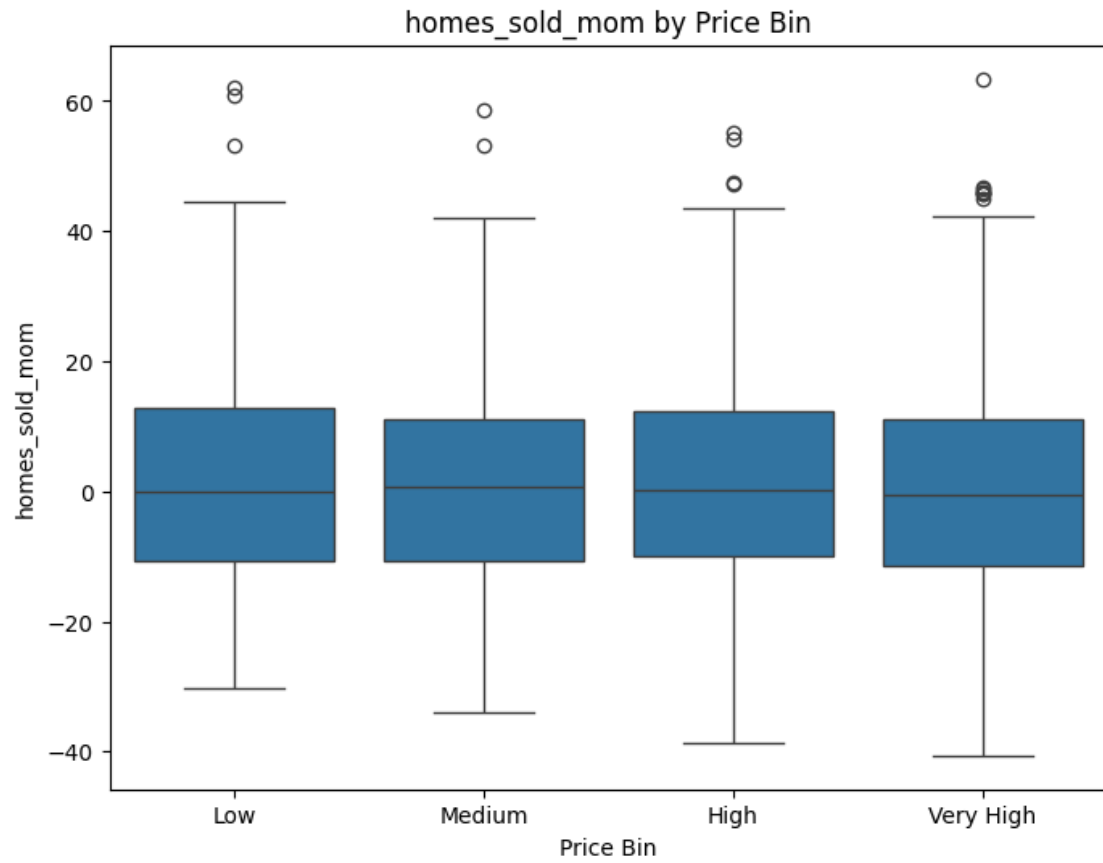
```
[9]: e.bivariate_boxplot(redfin_df, numerical_cols, categorical_cols)
```

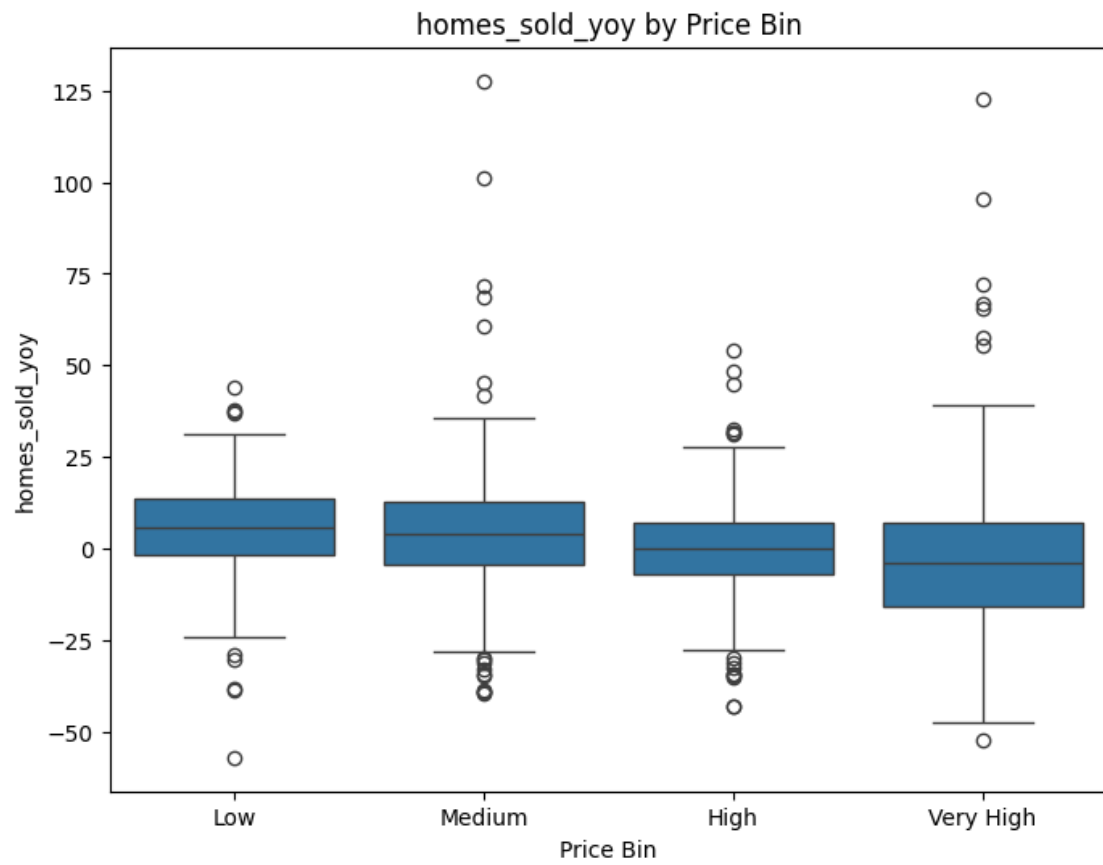


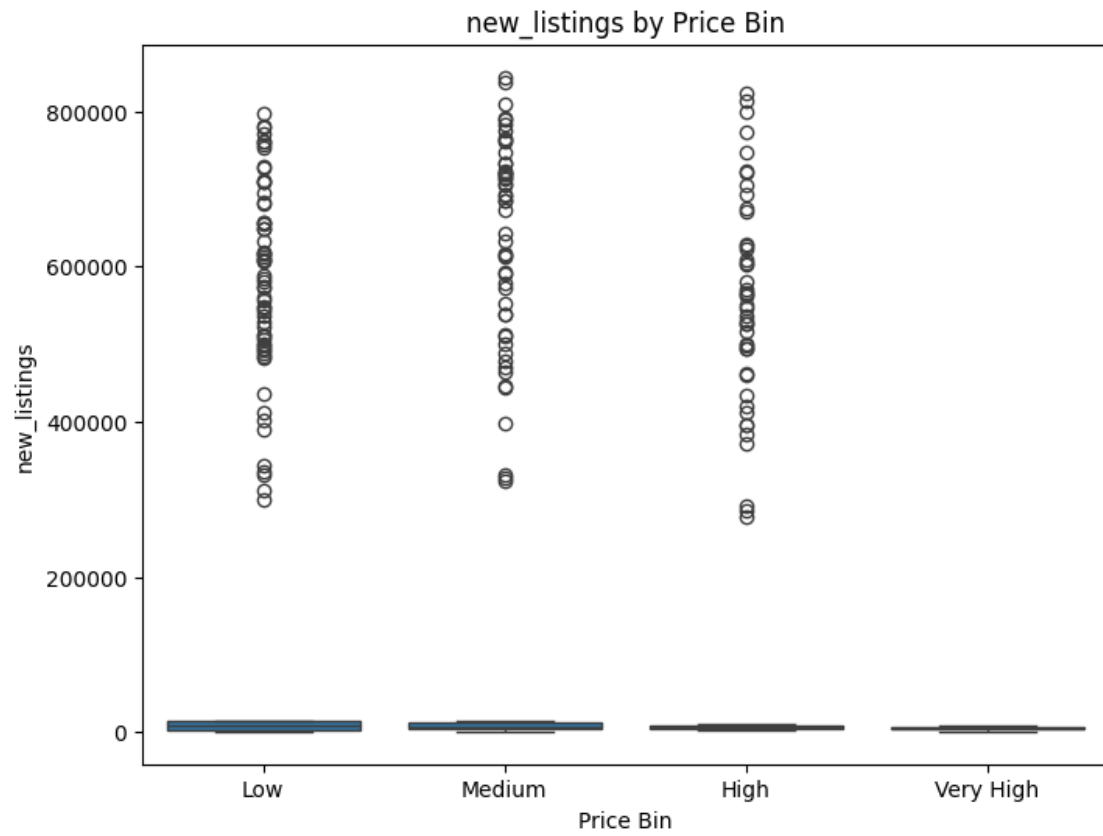


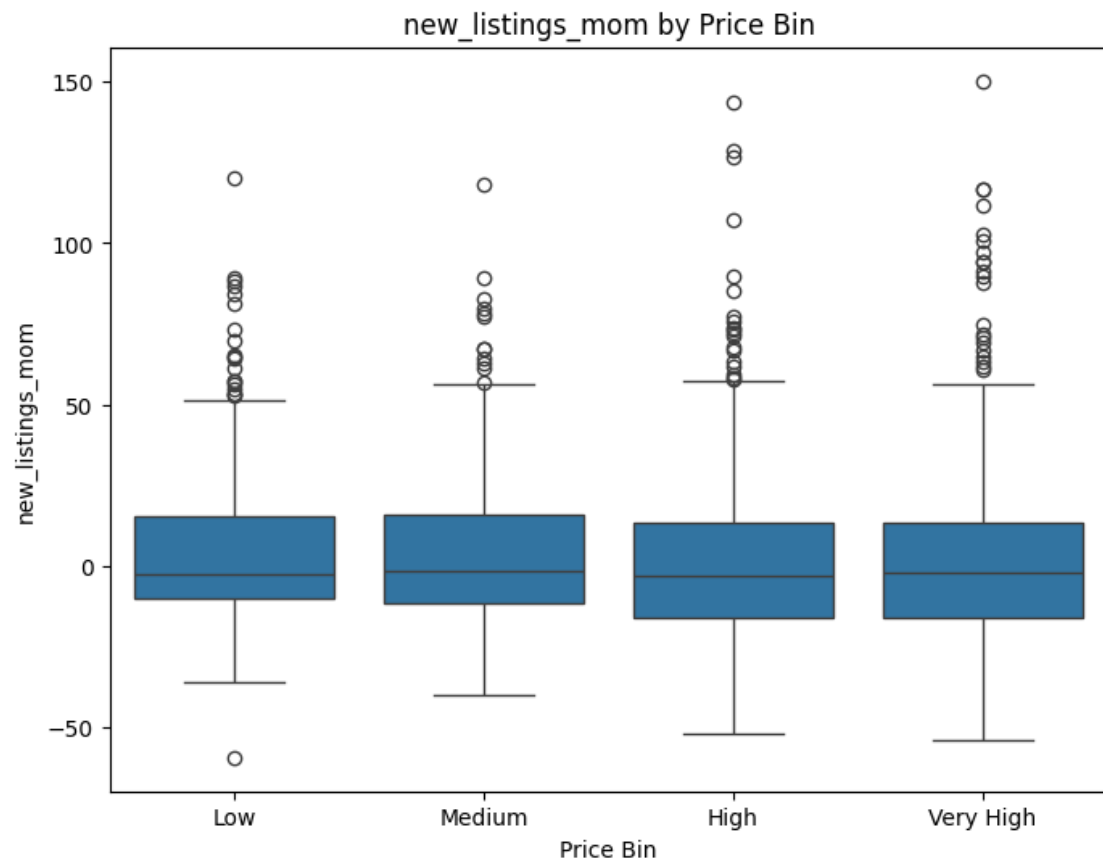


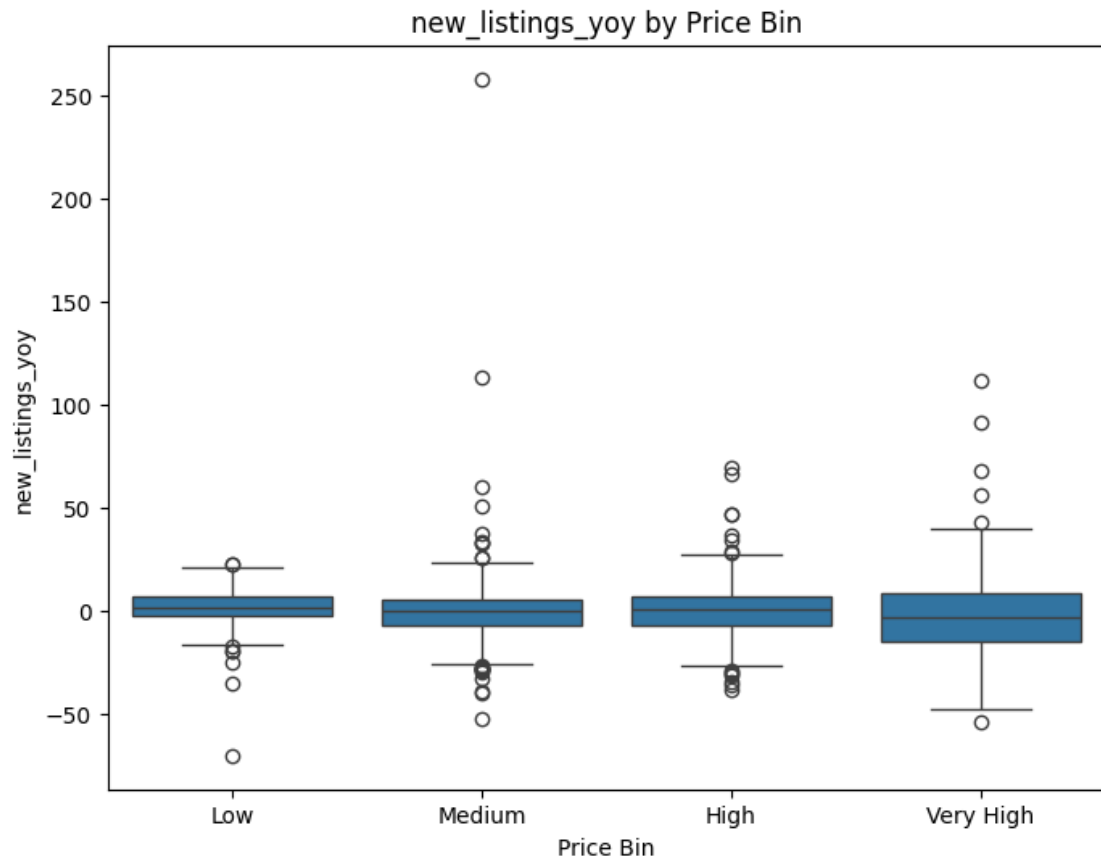


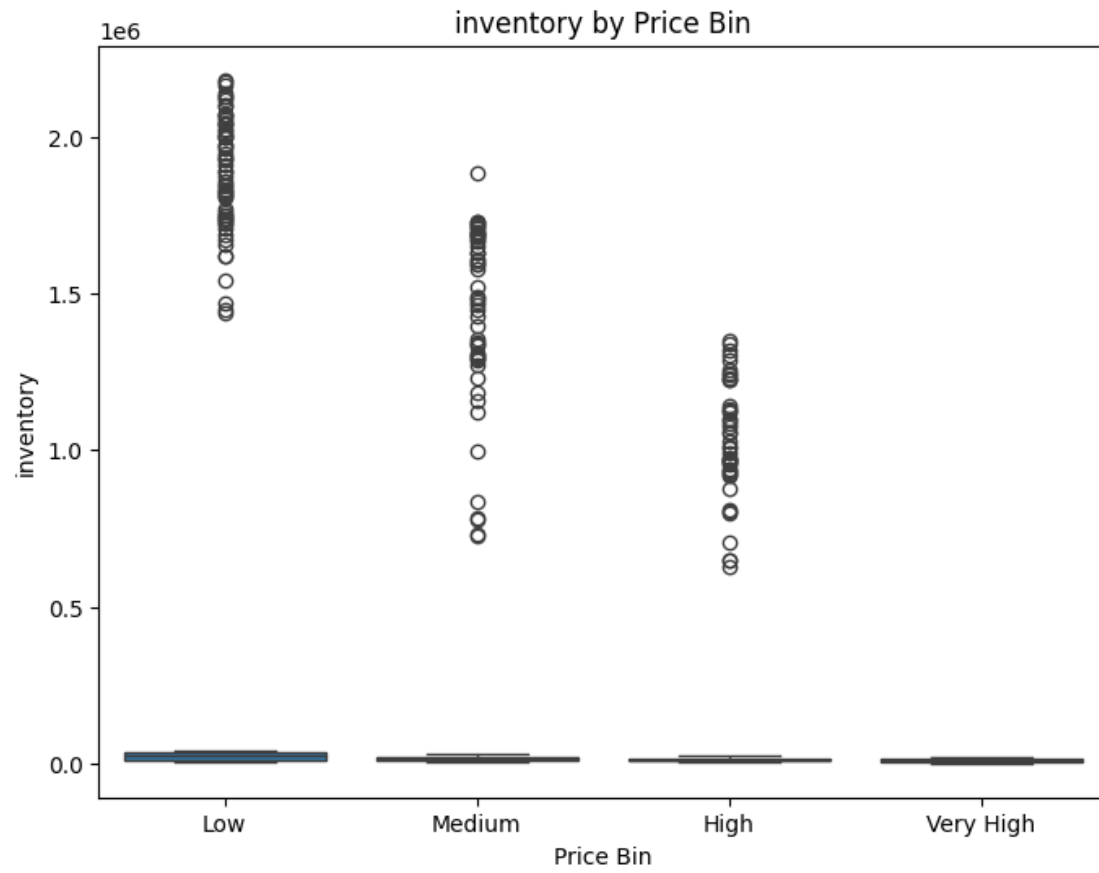


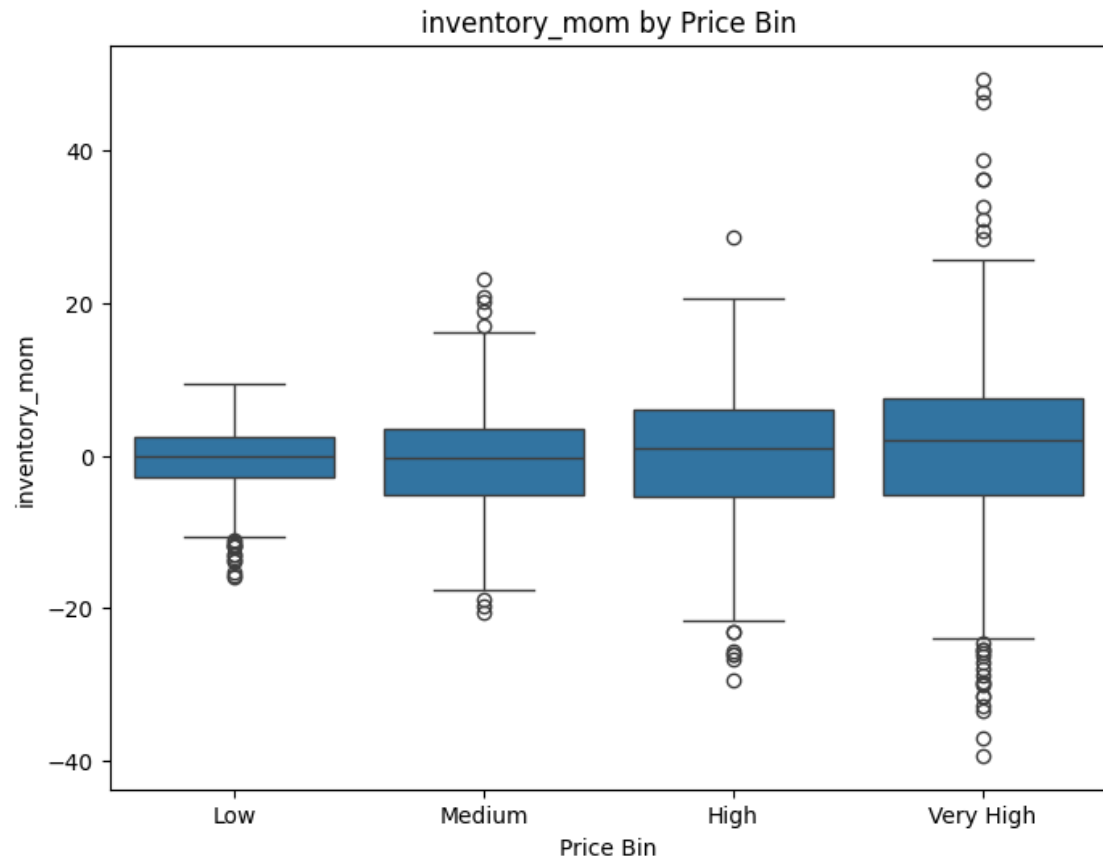


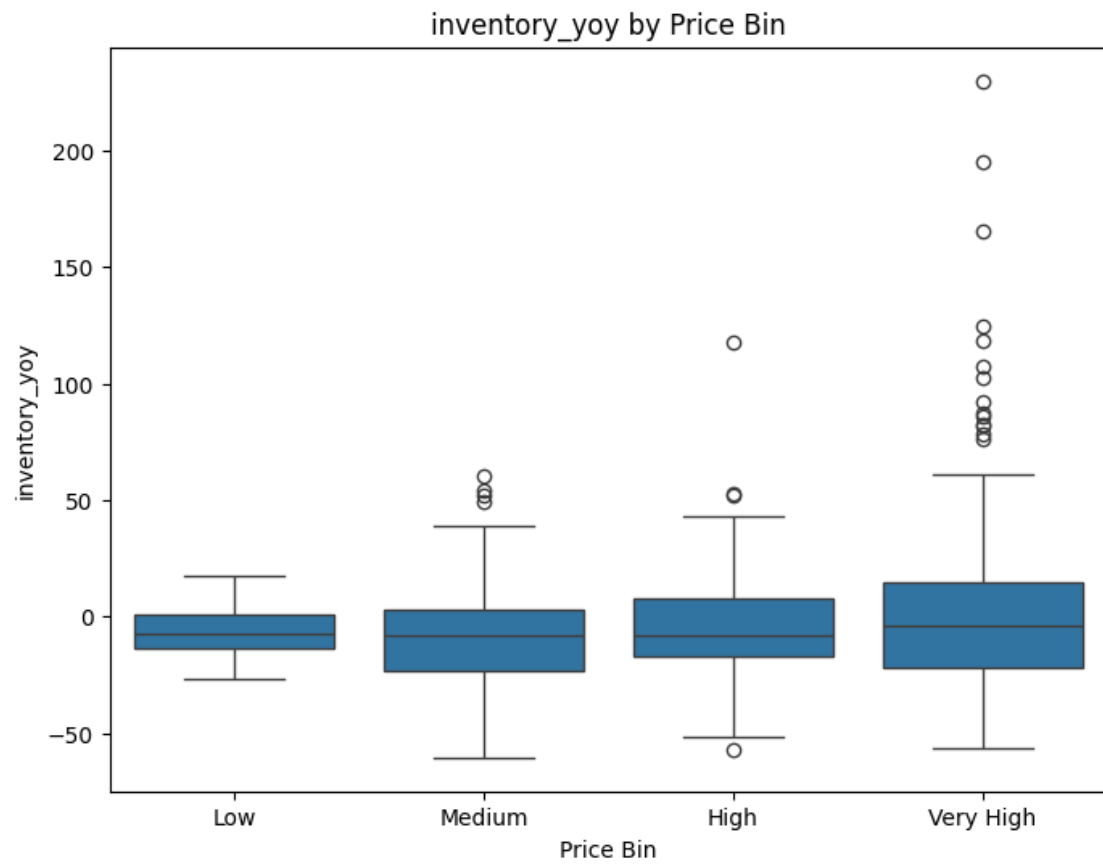


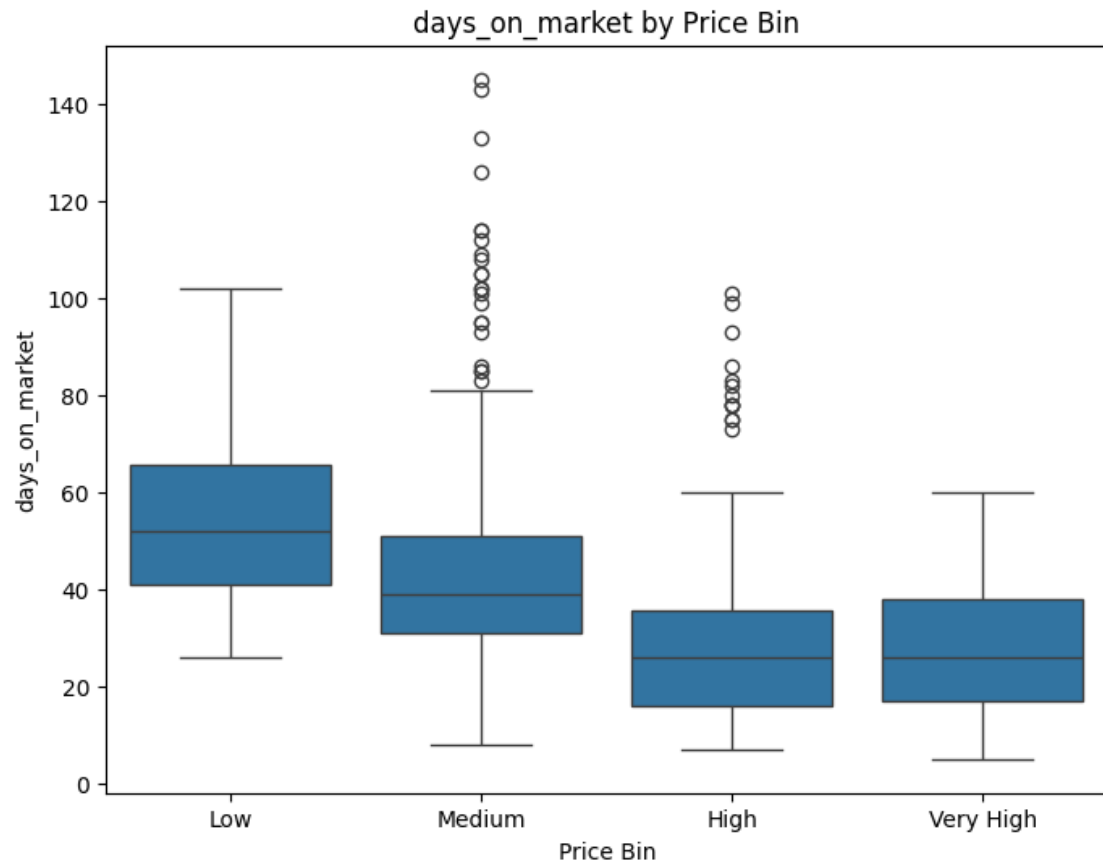


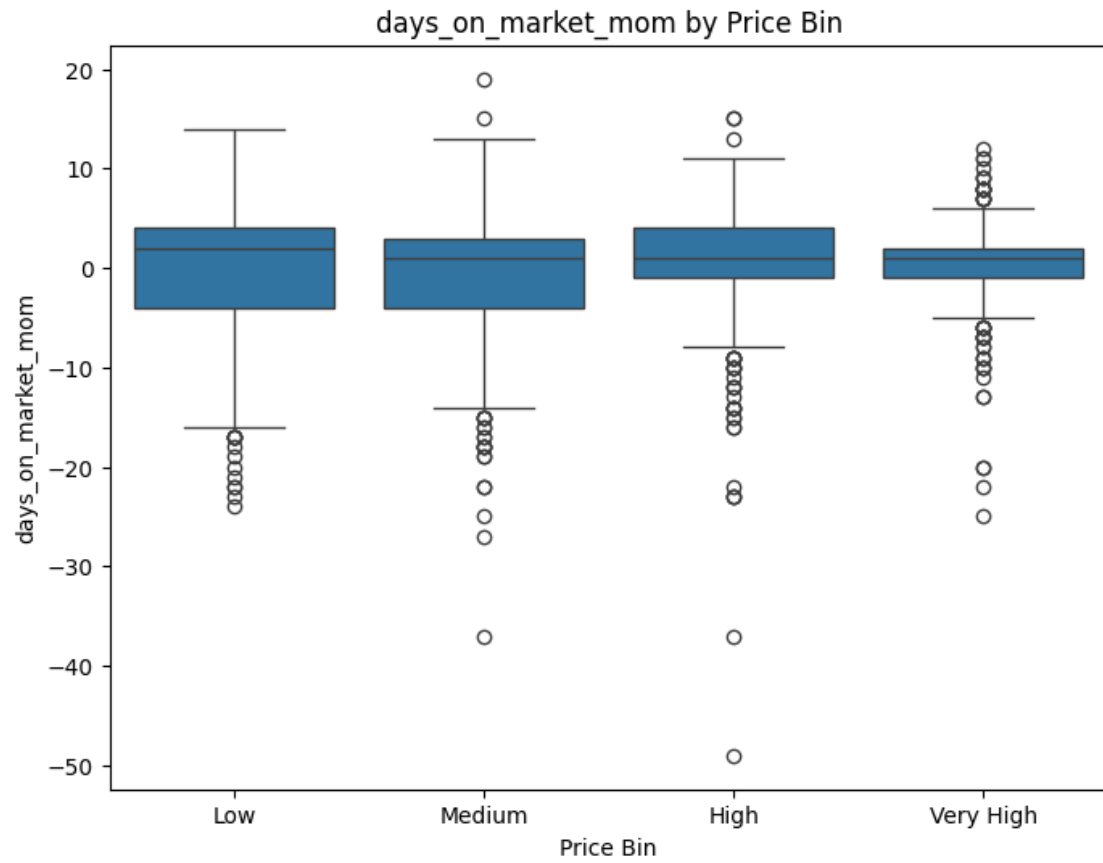


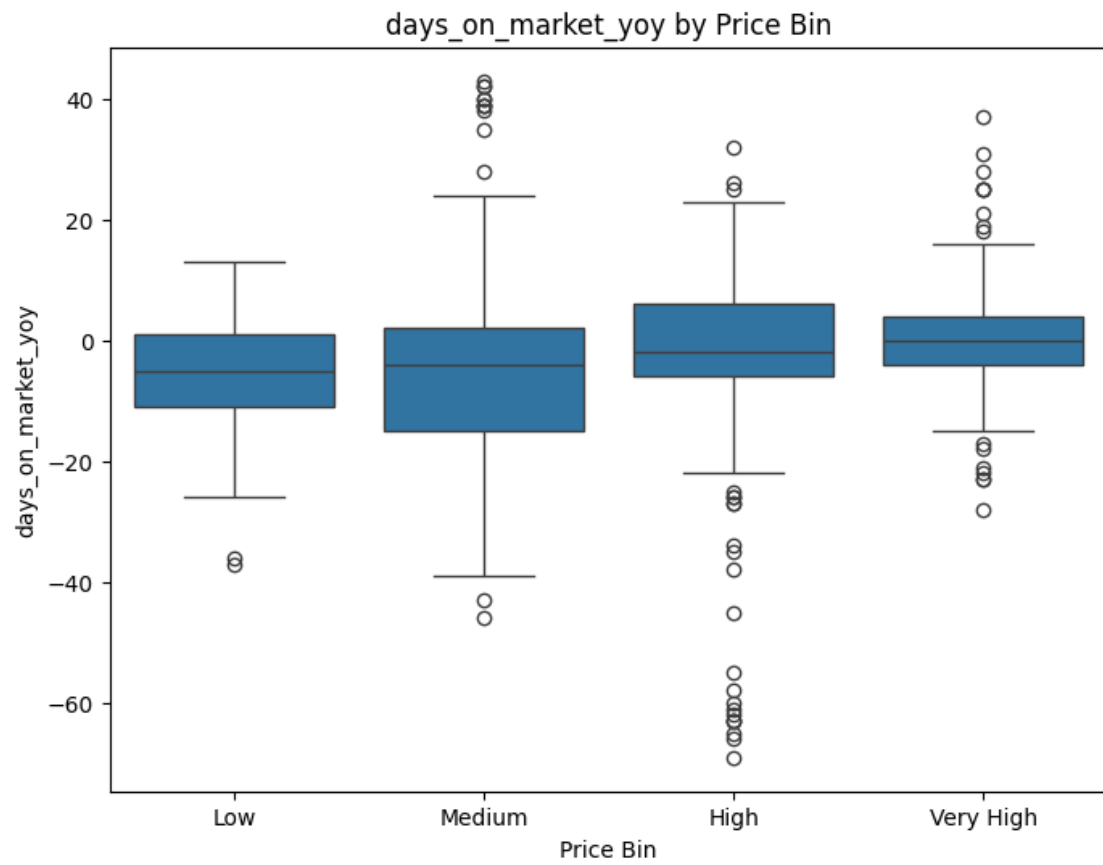


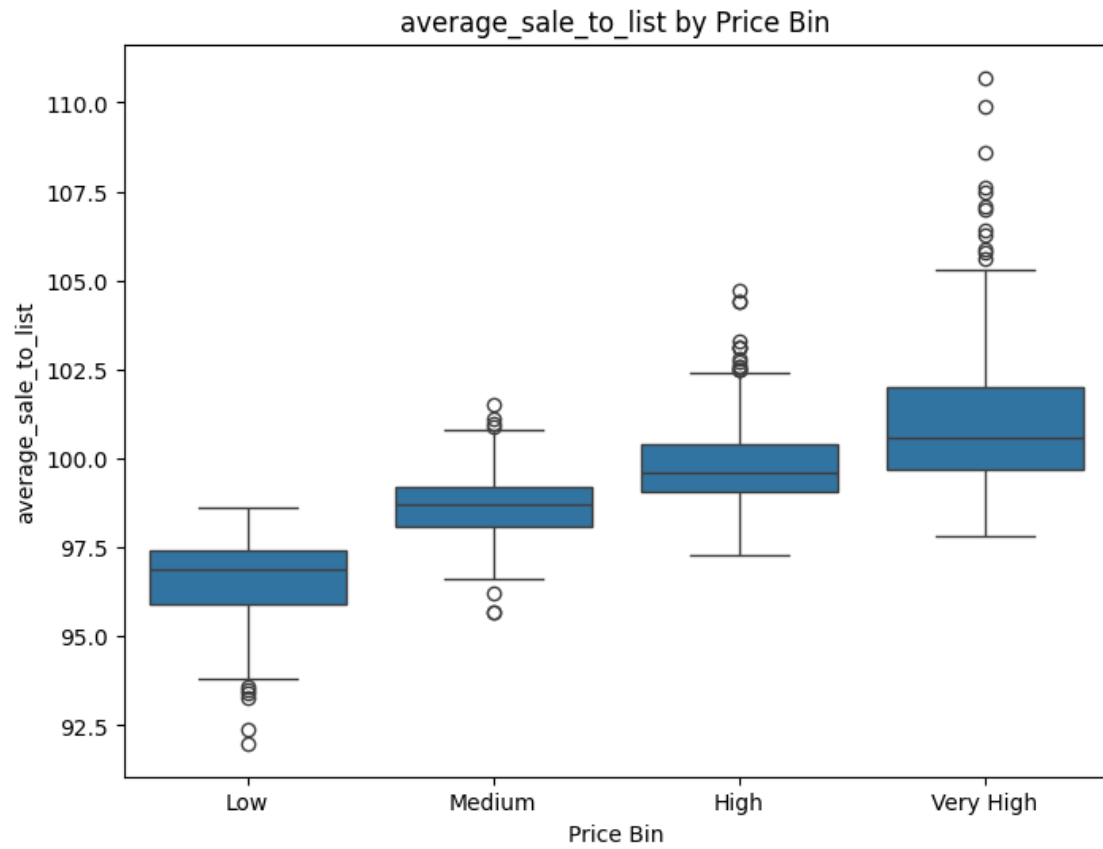


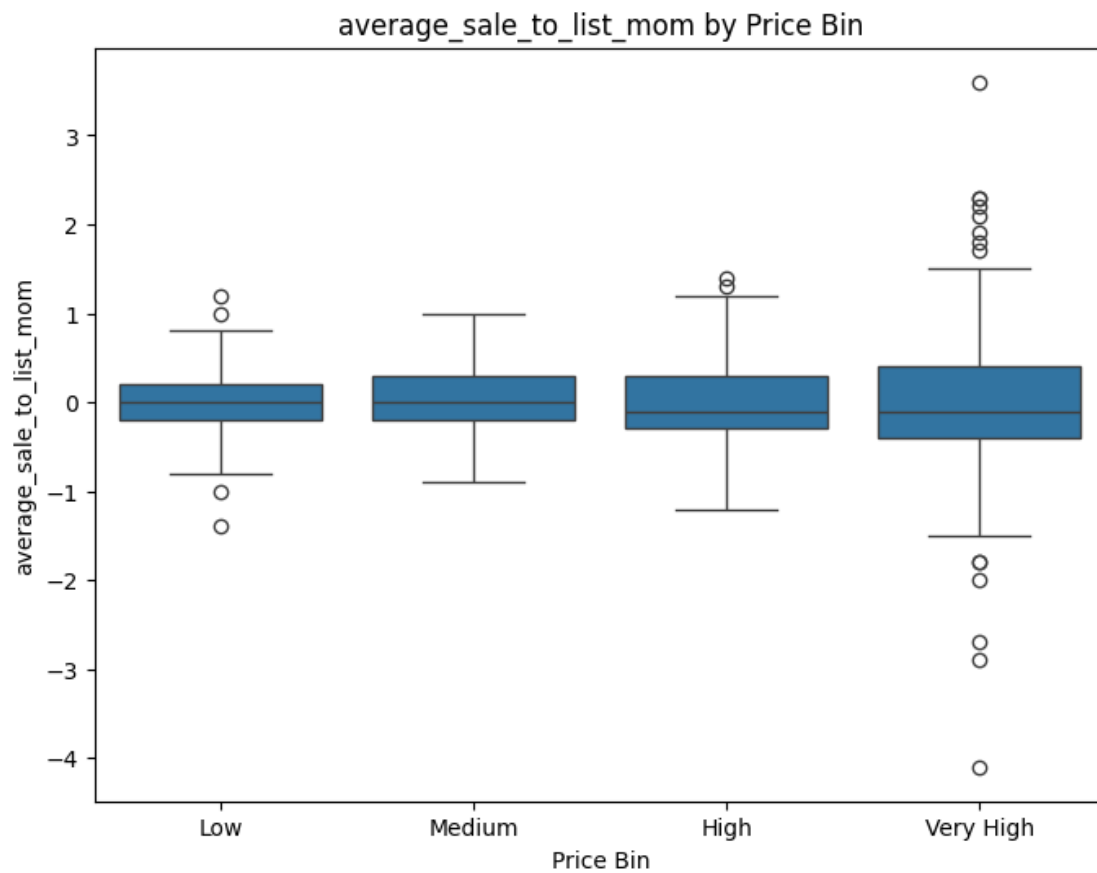


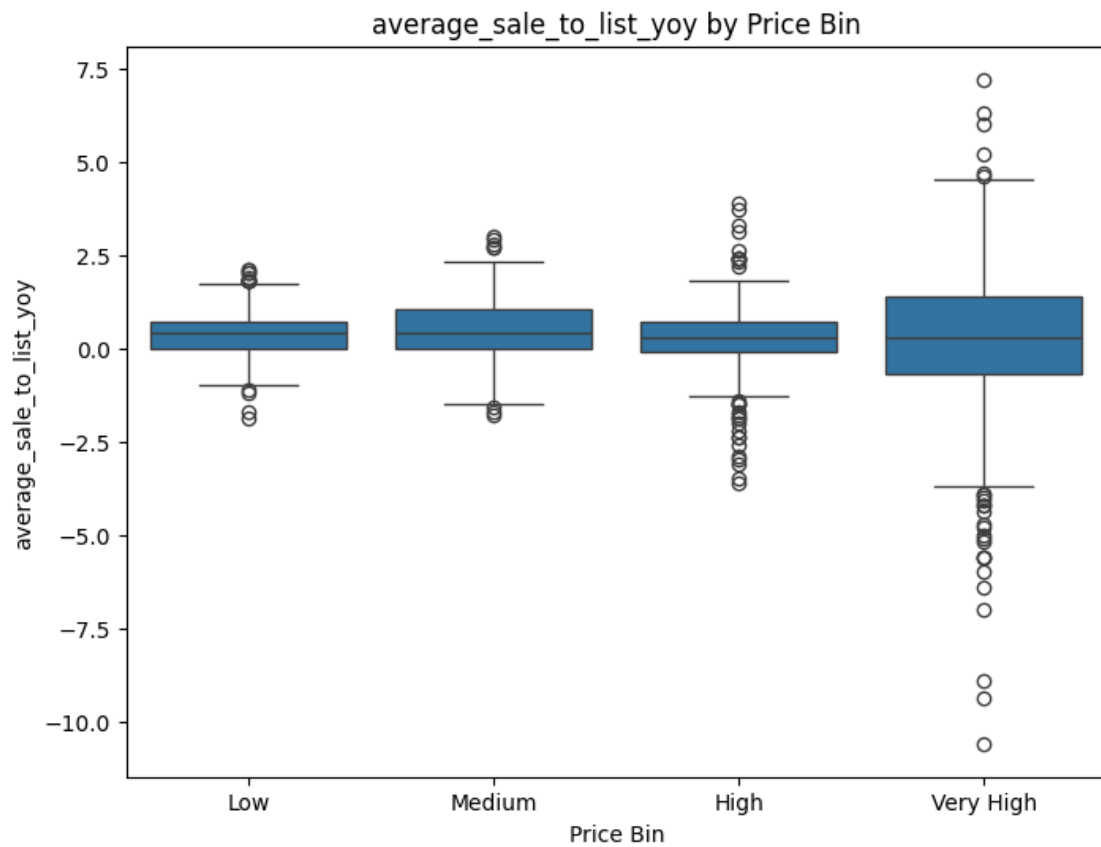


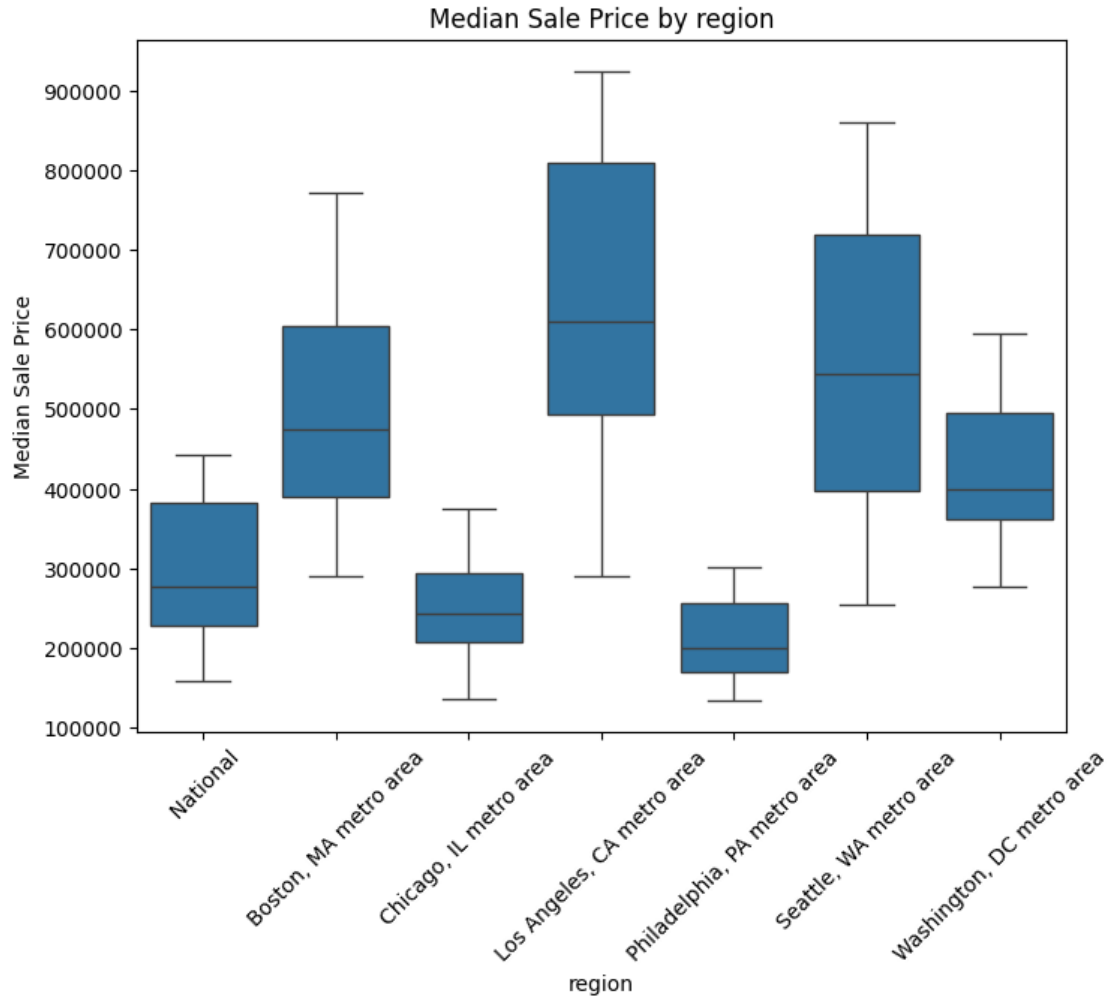












BoxPlot Insights

Below is an explanation of the key boxplots I've produced and why they matter for my project. Rather than listing every single variable, I'm focusing on those with notable outliers or high relevance to house-price modeling.

- Homes Sold (Boxplot) - Notable Outliers:** A few points stand far above the upper whisker, indicating especially high sales in certain months/regions. - **Relevance:** Large spikes in homes sold can influence average prices and reflect heightened demand. Such outliers may indicate unusually hot markets or seasonal booms (e.g., spring).
- Days on Market (Boxplot) - Outliers:** Some properties linger on the market significantly longer than typical, which can extend well beyond 100+ days. - **Relevance:** Days on market is inversely correlated with price—when houses sell quickly, prices are often higher. The outliers could represent unique property types, overpriced listings, or less desirable locations.
- Inventory (Boxplot) - Outliers:** A handful of months/regions show extremely high inventory levels, well above typical ranges. - **Relevance:** Excessive inventory can apply downward pressure on prices. Outlier months might coincide with shifts in economic conditions or new construction spikes.
- Median Sale Price (Boxplot) - Skew & High-Price Outliers:** The boxplot is right-skewed; some luxury or high-demand areas push prices well above the median. - **Relevance:** These outliers are integral to your analysis, because price is

the target variable. Log transformation or robust regressions might help handle these extremes in modeling. 5. Boxplot by price_bin - Why Binning?: Converting median_sale_price to categories (e.g., Low, Medium, High) allows us to see how other metrics (like homes_sold, days_on_market) differ across price tiers. - Key Observations: Higher-tier price bins often correlate with fewer days on market and slightly lower inventory, suggesting strong demand in more expensive markets. A small subset of extreme outliers can appear in each bin (e.g., unusually fast or slow sales).

Next Steps

- Decide on Outlier Treatment: For modeling, should I exclude or transform extreme values?
- Connect Boxplots to Time-Series: Check whether outlier months line up with anomalies in my seasonal decomposition or other metrics (e.g., interest rate changes).
- Refine Feature Selection: Outliers can distort correlations. I will consider robust scaling or transformations (log, sqrt) to reduce skew.

Research Questions

To better structure our analysis and ensure alignment with our research goals, we explore the following key questions:

1. What factors most influence median sale prices?
Rationale: Understanding the drivers behind median sale prices is critical for building predictive models and identifying key variables.
2. How do changes in inventory levels affect future prices?
Rationale: Inventory fluctuations could be leading indicators of price changes, allowing investors to make better decisions.
3. Are seasonal trends significant in price fluctuations?
Rationale: Identifying recurring patterns in the housing market can improve forecasting accuracy, particularly for short-term price movements.
4. Can we predict price changes using historical data?
Rationale: The core goal of this project is to assess whether past data can reliably forecast future trends, supporting the hypothesis that housing market shifts are predictable using machine learning techniques.

By structuring our analysis around these questions, we ensure that our data exploration, feature engineering, and modeling phases remain focused and aligned with the research objectives.

Research Question #1

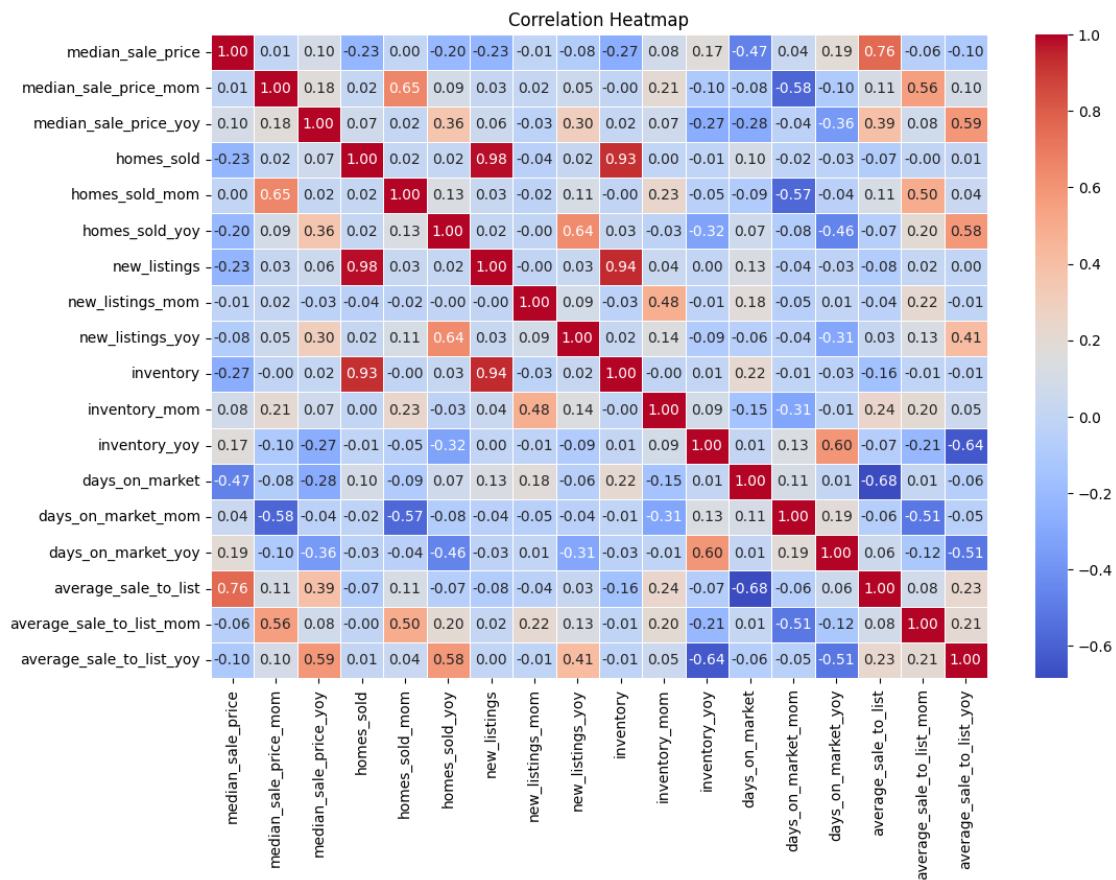
1. What factors most influence median sale prices?
 - We will use a correlation heatmap to identify the strongest relationships between median_sale_price and other variables.

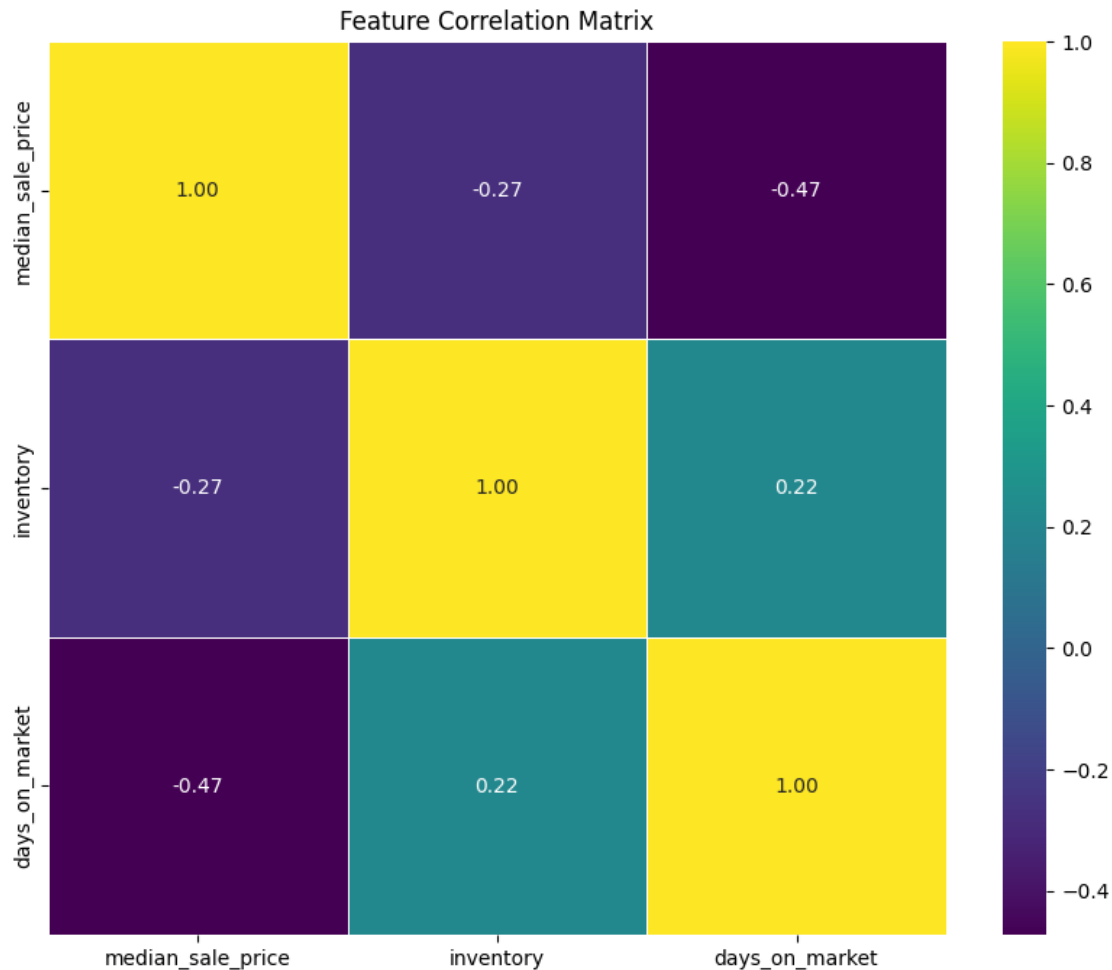
```
[10]: # Correlation heatmap for all columns
e.plot_correlation_heatmap(redfin_df)

cols_of_interest = ['median_sale_price', 'inventory', 'days_on_market']
```

```
# Select only numeric columns from the subset
redfin_df_subset = redfin_df[cols_of_interest].select_dtypes(include=['number'])

# Now call the function
e.plot_feature_correlation_heatmap(redfin_df_subset, figsize=(10, 8),
    cmap='viridis')
```





Question #1 Insights

What Factors Most Influence Median Sale Prices?

From our correlation heatmap, the strongest relationships with median_sale_price are:

1. Days on Market (-0.47 correlation)
 - Homes tend to sell faster when prices are higher, meaning that a shorter time on the market is associated with higher sale prices.
 - This suggests that competitive markets with high demand lead to quicker sales at higher prices.
2. Inventory Levels (-0.27 correlation)
 - Higher inventory levels correlate with lower prices, indicating that an oversupply of homes may push prices down.
 - This aligns with fundamental supply-demand principles in real estate markets.
3. Homes Sold & New Listings (Positive Correlation)
 - More homes sold and higher new listings are associated with rising prices.
 - A strong market with increased transaction activity often signals a growing demand that supports higher prices.

4. Sale-to-List Price Ratio (Moderate Positive Correlation)
 - Higher sale-to-list ratios coincide with stronger market conditions, where homes sell closer to or above their listing price.
 - This suggests that buyers are willing to pay more in competitive markets, leading to higher median sale prices.
5. Month-over-Month (MoM) Correlations
 - Some MoM metrics (e.g., `homes_sold_mom` and `median_sale_price_mom`) show moderate correlations, indicating that monthly changes in sales can predict short-term price shifts.

Interpreting High Correlation Values:

- Close to +1: Two features move together almost every time (e.g., `new_listings` vs. `inventory`, where more listings generally mean more available inventory).
- Close to -1: A strong inverse relationship (e.g., higher `median_sale_price` corresponds to fewer `days_on_market`).
- Close to 0: No strong linear relationship, though nonlinear interactions may still exist.

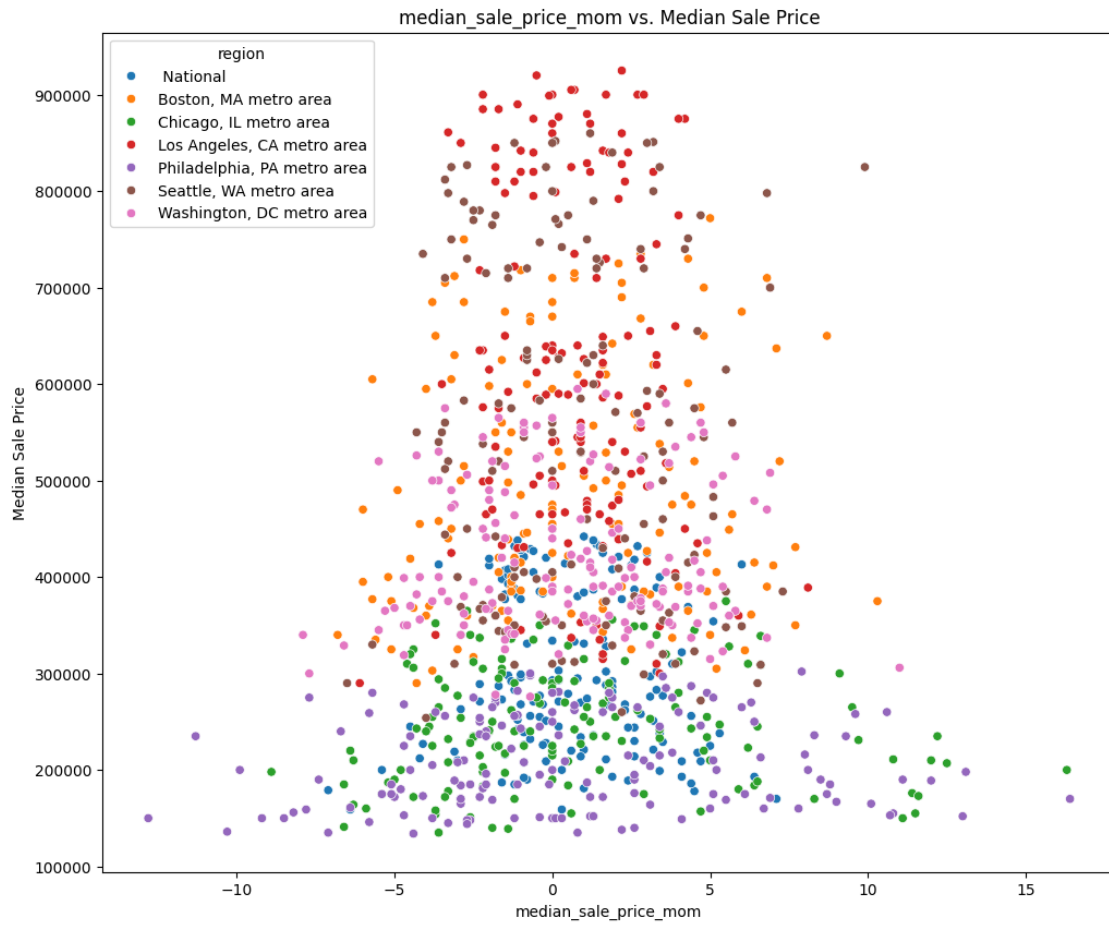
Caveats & Next Steps:

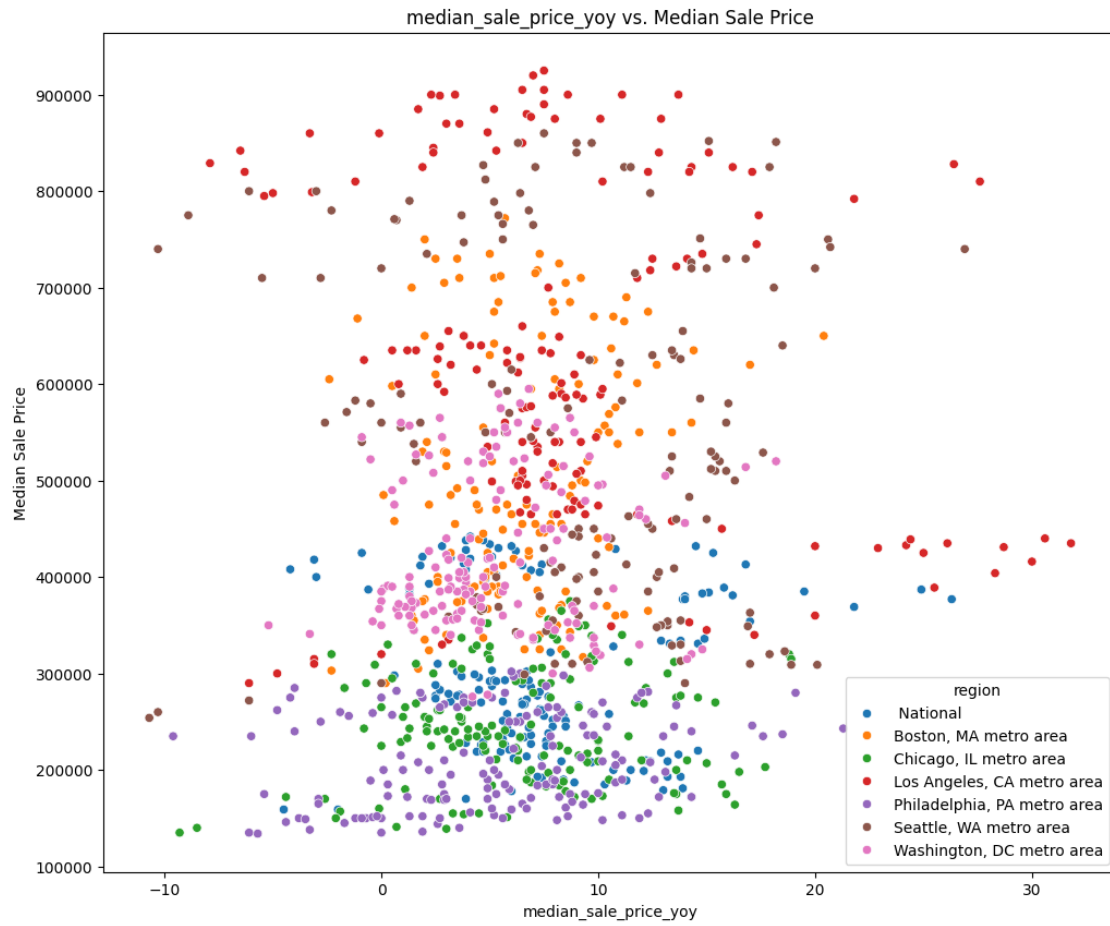
- Correlation ≠ Causation: Even though `median_sale_price` and `inventory` are correlated, external factors like interest rates and economic conditions could influence both.
- Outliers & Nonlinearities: Some relationships might be distorted by outliers—further scatterplot analysis and transformations (e.g., log scaling) may help refine insights.
- Modeling Implications: Highly correlated variables can introduce multicollinearity in regression models. We may need feature selection techniques like Ridge/Lasso regression to mitigate this issue.

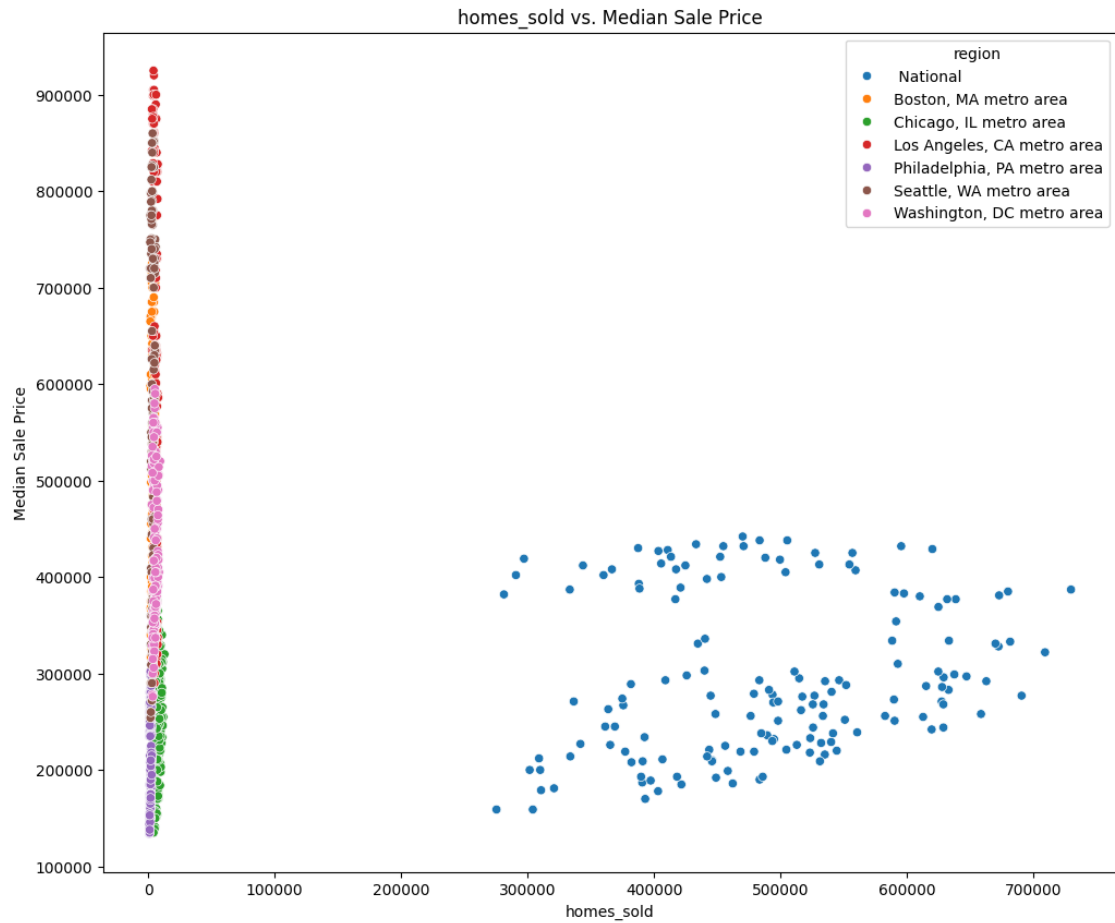
Research Question #2

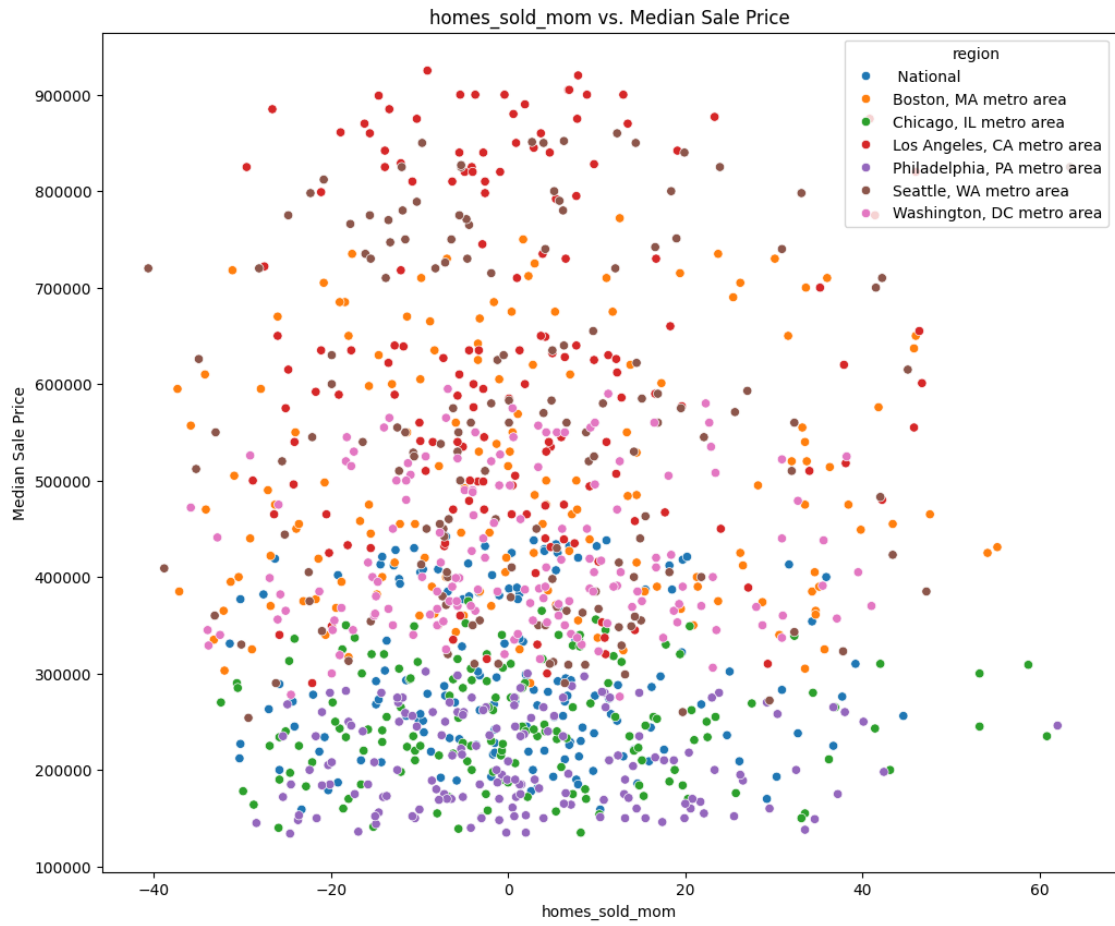
2. How do changes in inventory levels affect future prices?
 - We'll create a scatter plot comparing `inventory` and `median_sale_price`.

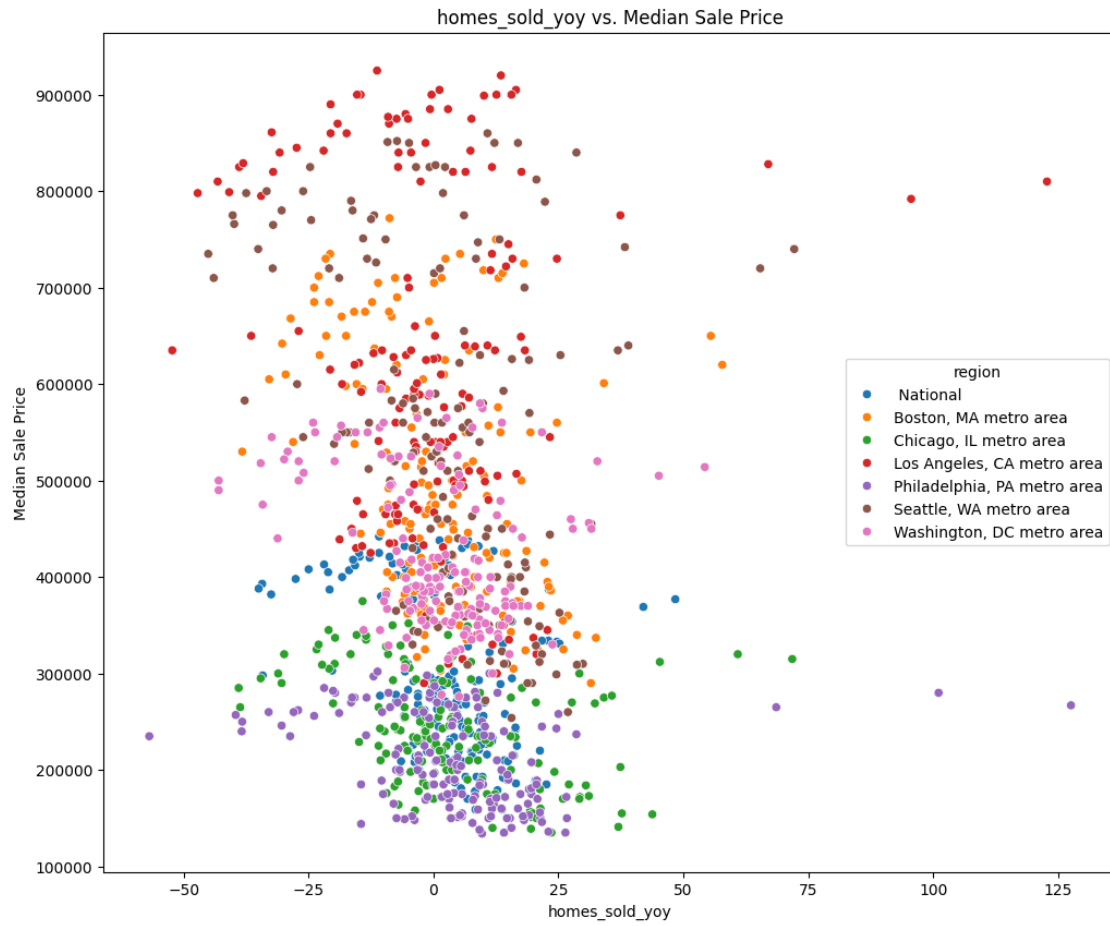
```
[11]: e.bivariate_scatterplot(redfin_df, numerical_cols, categorical_cols)
```

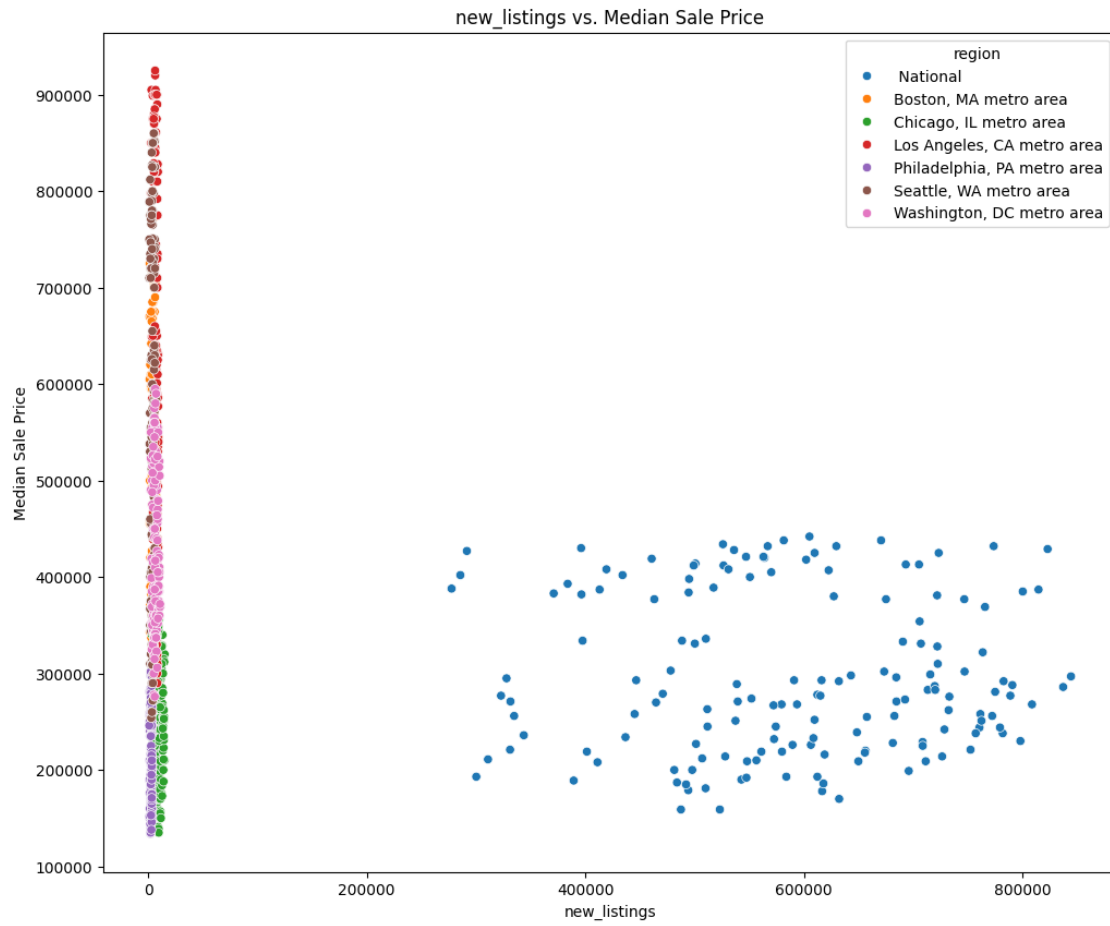



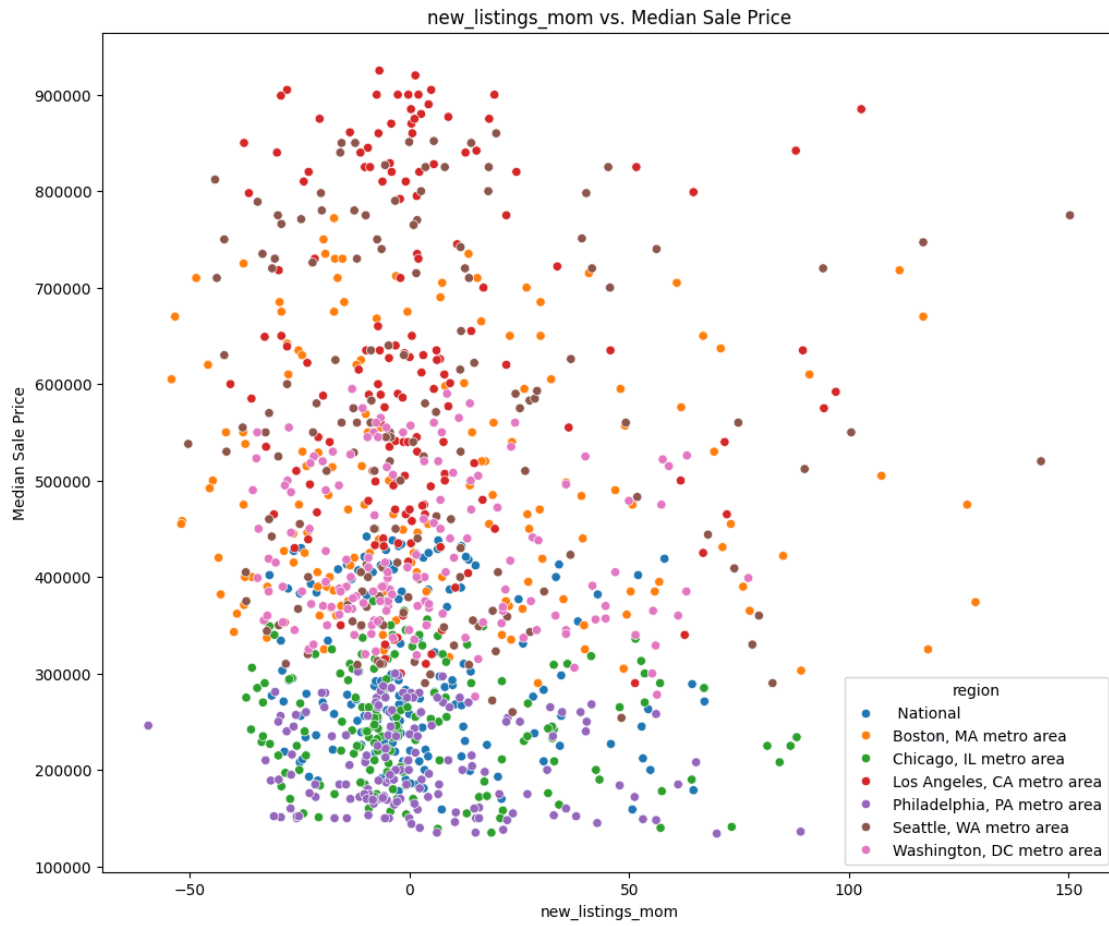


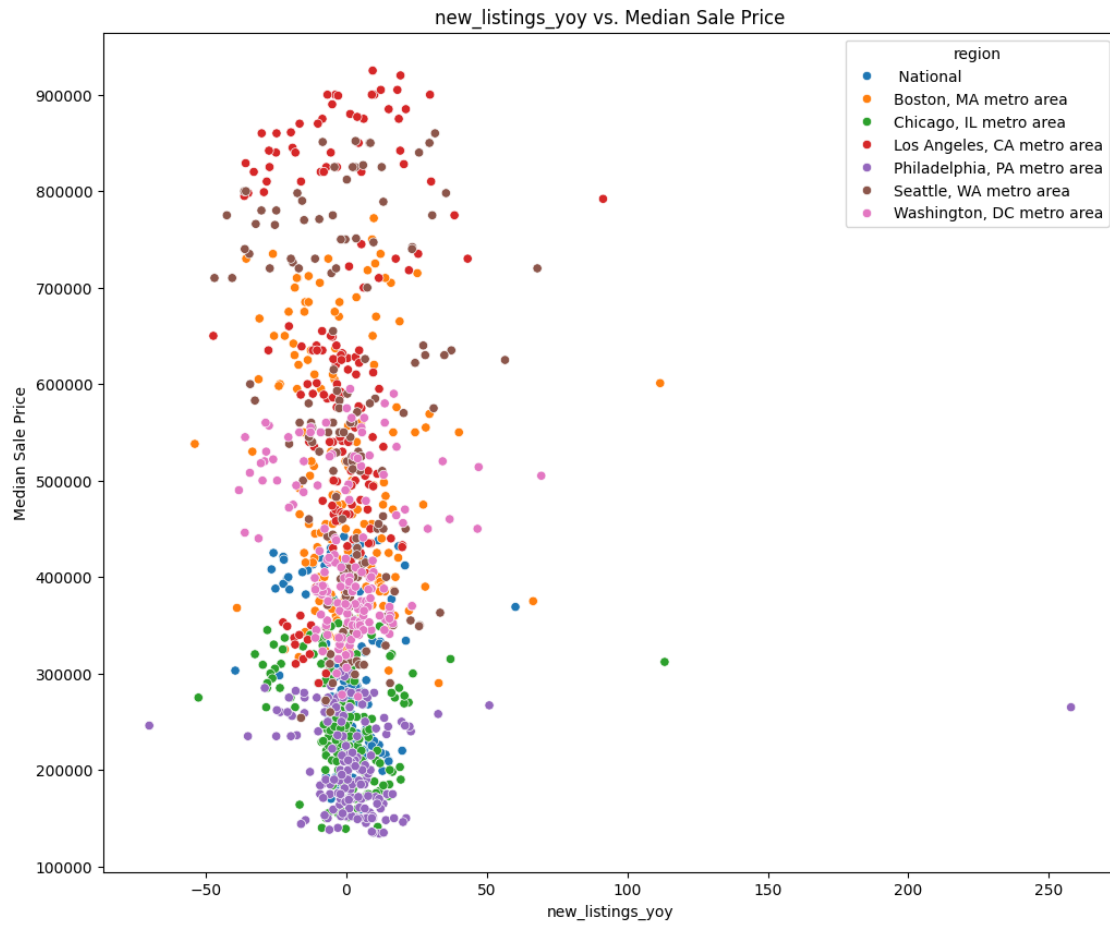


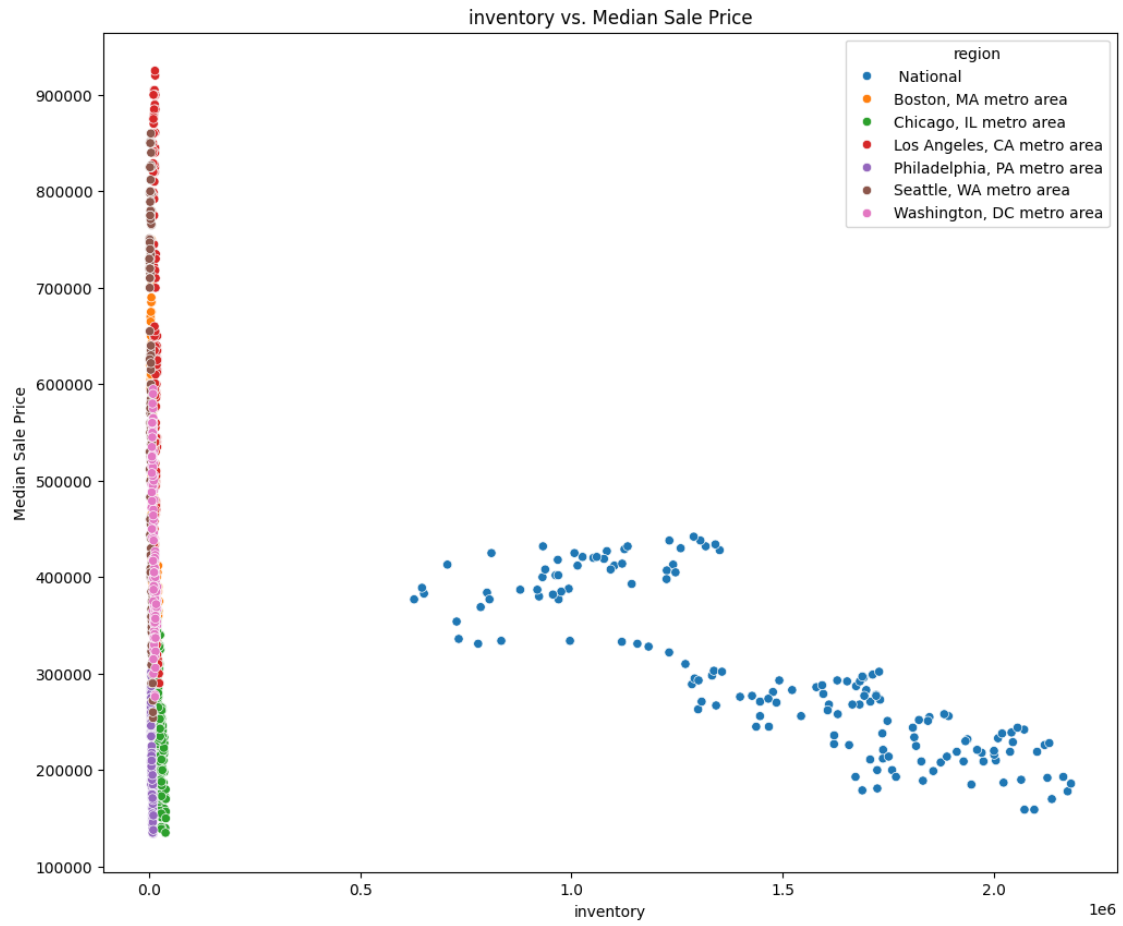


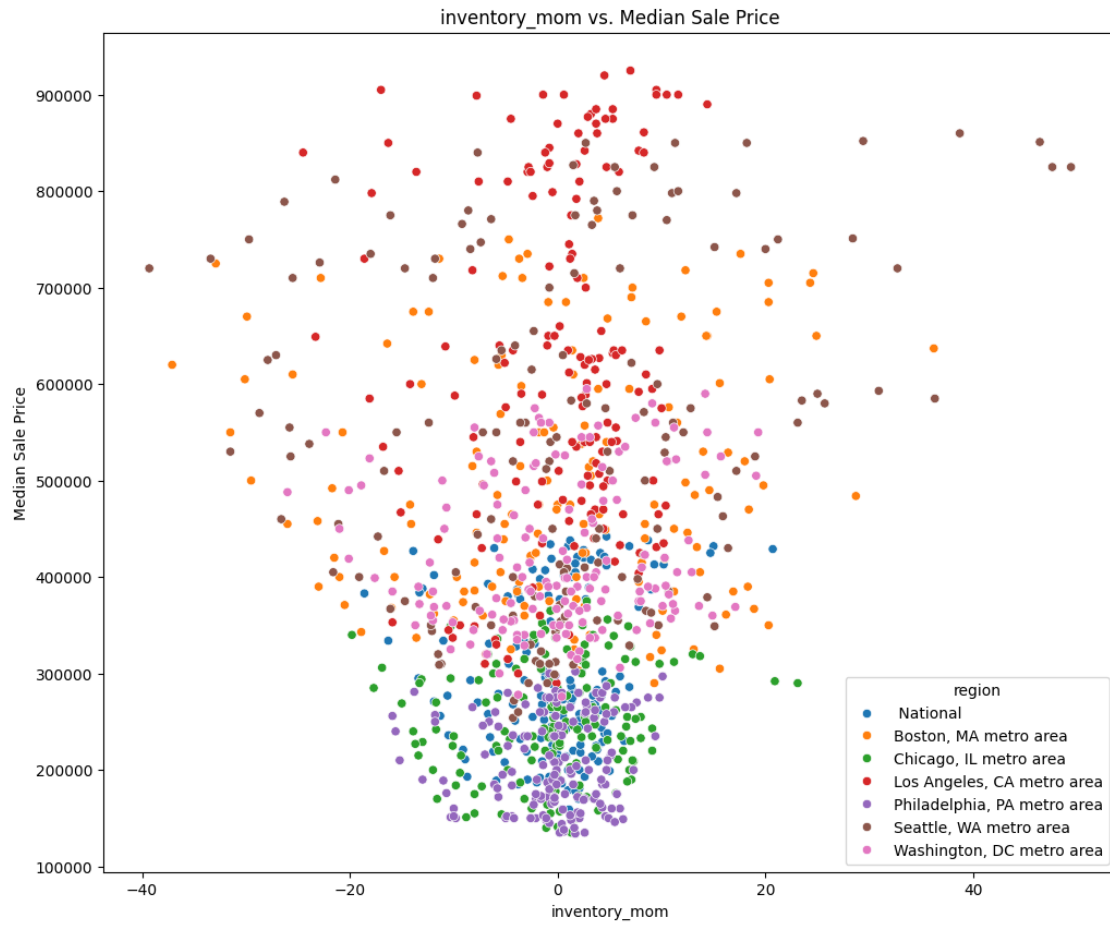


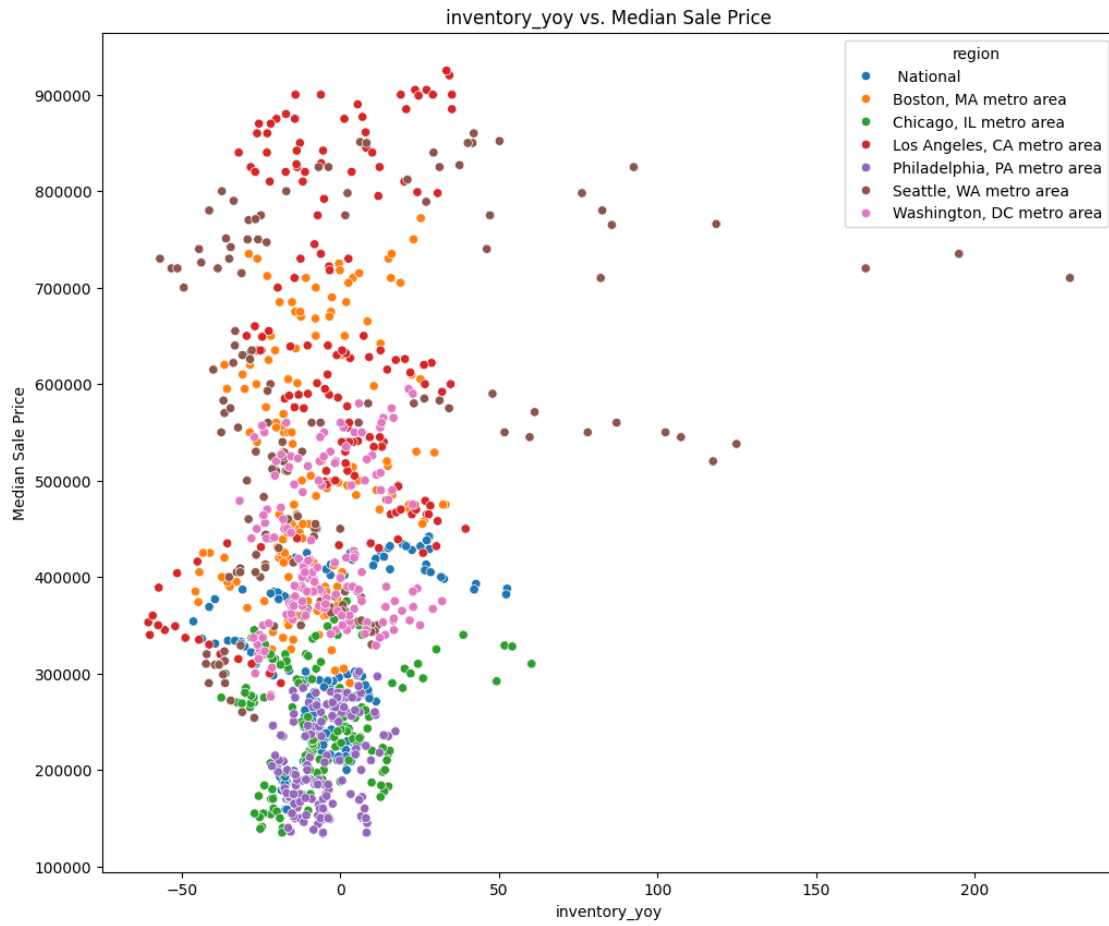


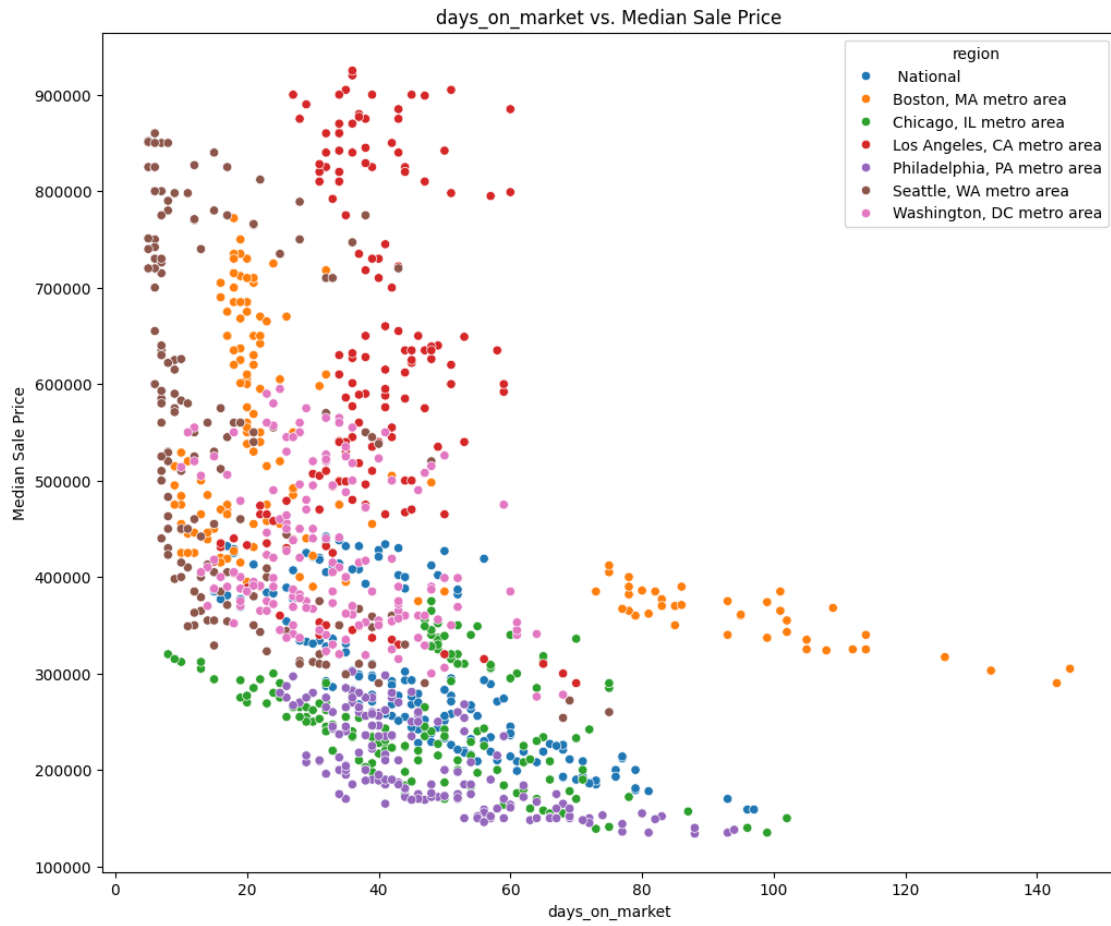


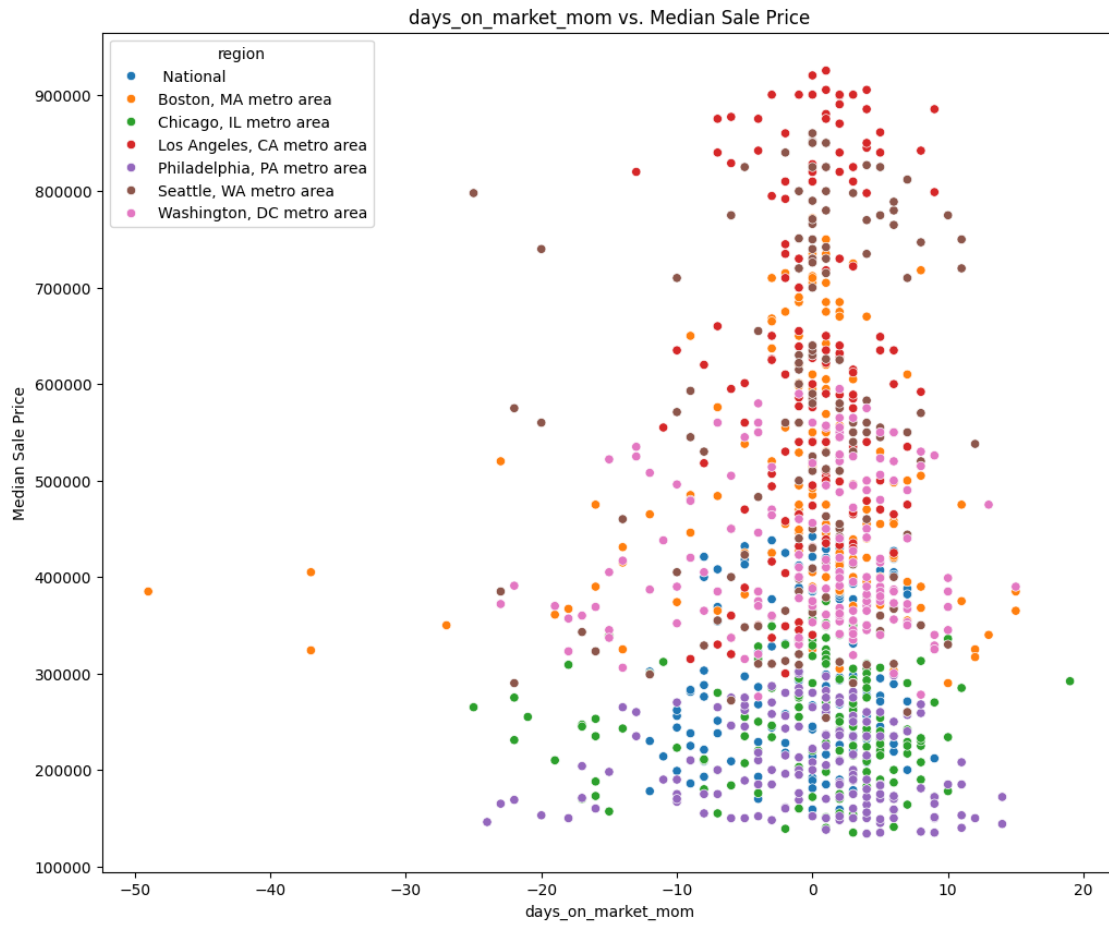


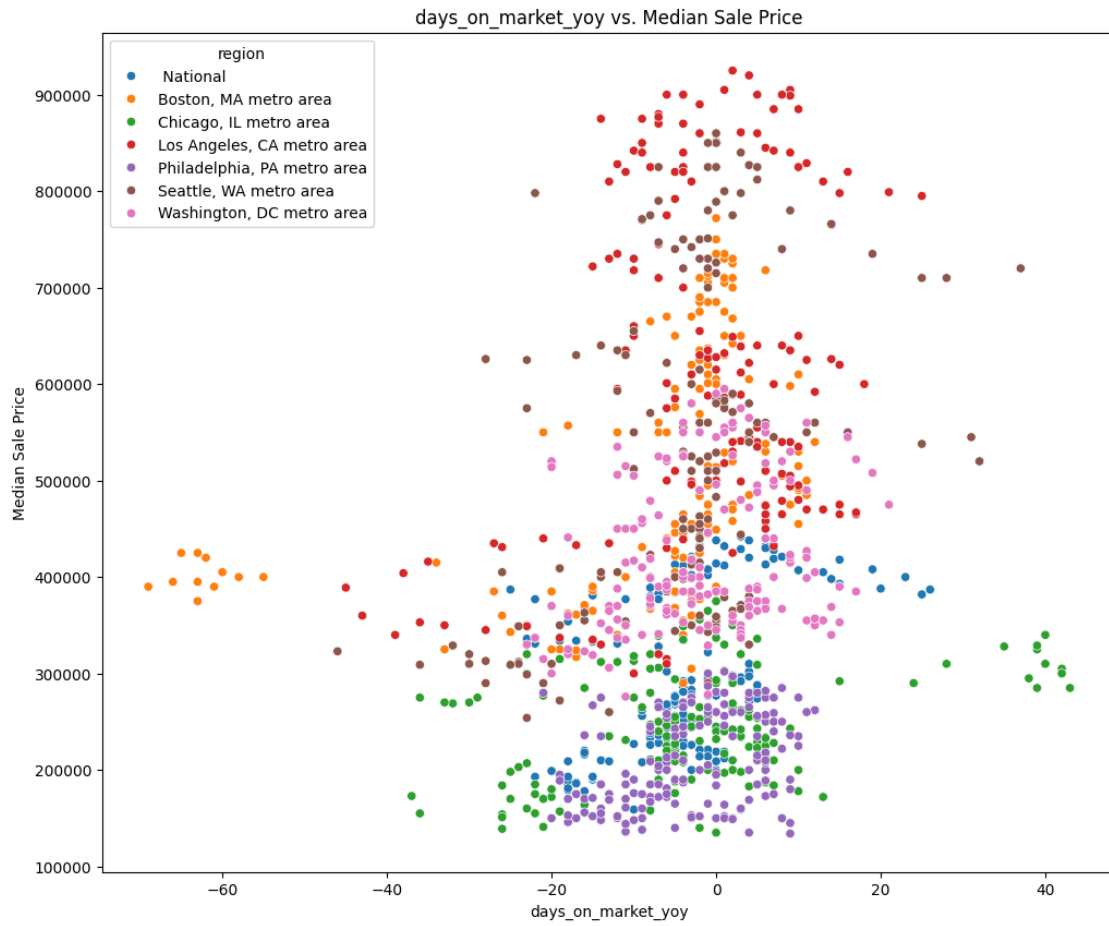


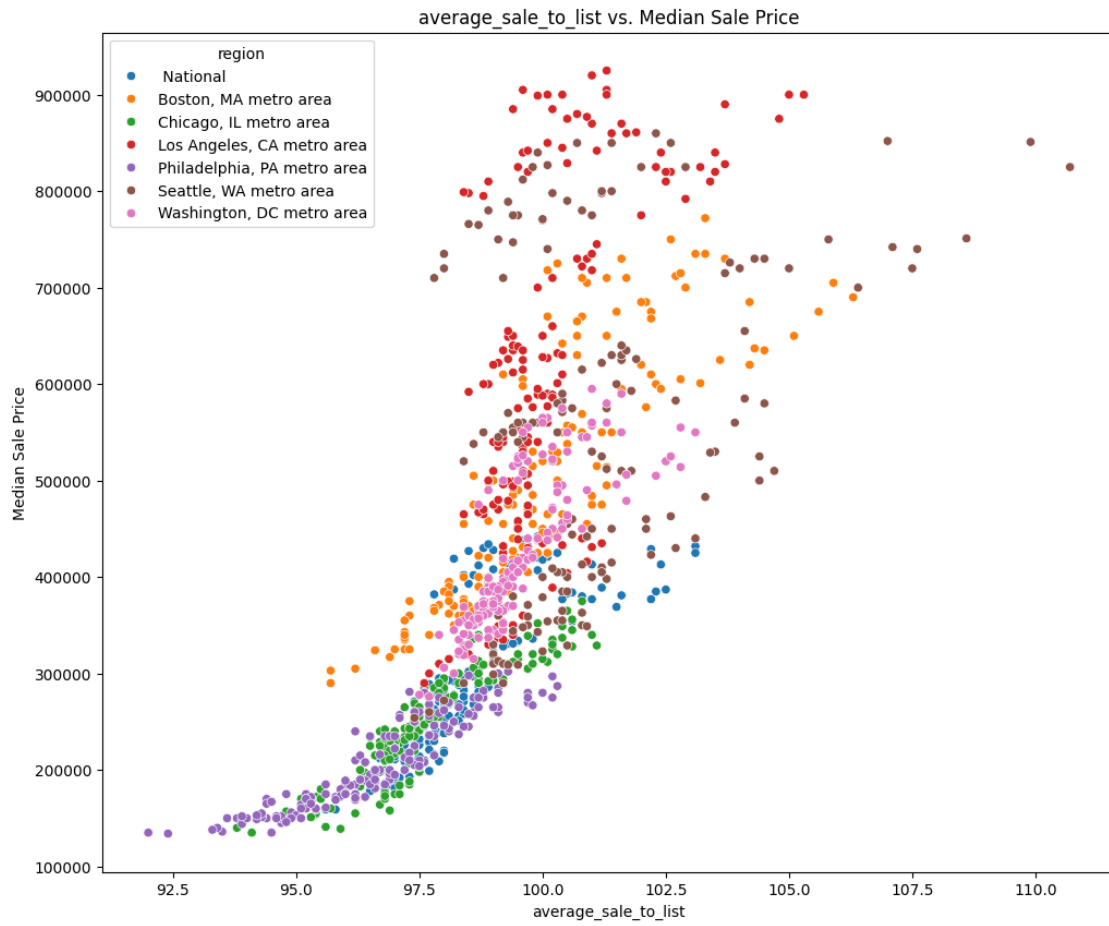


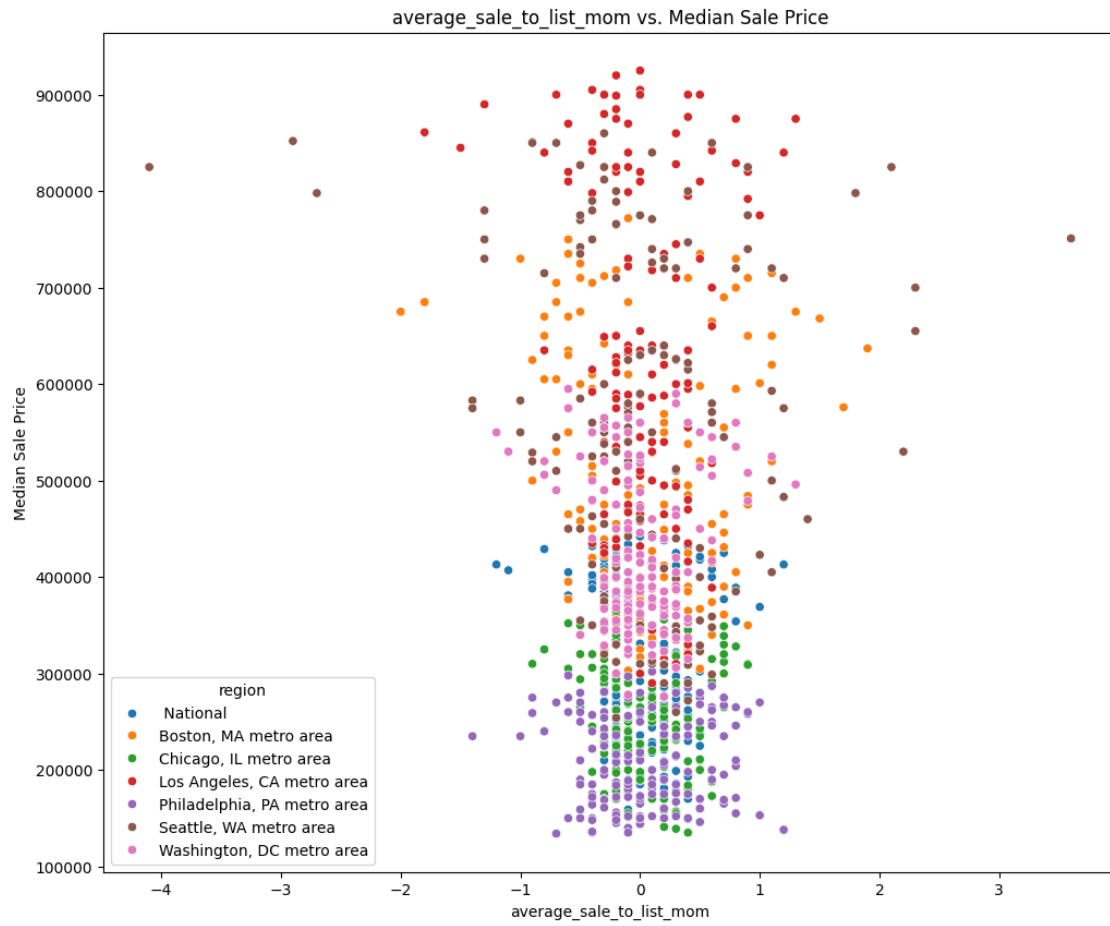














Question #2 Insights

How Do Changes in Inventory Levels Affect Future Prices?

To explore this question, we conducted a **bivariate scatterplot analysis** comparing **median sale prices** against various market features. This allowed us to not only investigate inventory but also observe broader market interactions.

Key Findings from the Scatterplots

- Inventory vs. Median Sale Price (Negative Correlation)** - Higher inventory levels **tend to coincide with lower prices**. - This follows economic supply-demand principles—when there are more homes available, buyers have more negotiating power, often leading to **lower sale prices**. - However, the effect is not immediate, suggesting a **lag between inventory increases and price reductions**.
- Lag Effect: Inventory Changes Precede Price Movements** - In markets like **Seattle and National trends**, an **increase in inventory does not immediately decrease median prices**. - Instead, price shifts often **lag behind inventory changes by a few months**. - This suggests that **inventory could serve as an early warning indicator** for future price adjustments.
- Regional Market Differences** - Some markets **resist price declines despite rising inventory**. For example:
 - **Los Angeles (Red)**: High-priced properties remain expensive despite fluctuations in available inventory.
 - **Boston (Orange)**: Shorter days on market indicate demand is keeping up with supply, reducing the downward pressure on prices.
 - The **National market (Blue)** smooths out these

fluctuations, reinforcing that **local trends must be analyzed separately**. 4. **Sales Volumes and Market Competitiveness** - A high number of **homes sold** in certain regions keeps prices stable despite inventory shifts. - In **Washington, DC, and Philadelphia**, strong buyer activity appears to offset inventory-driven price drops, keeping values more consistent.

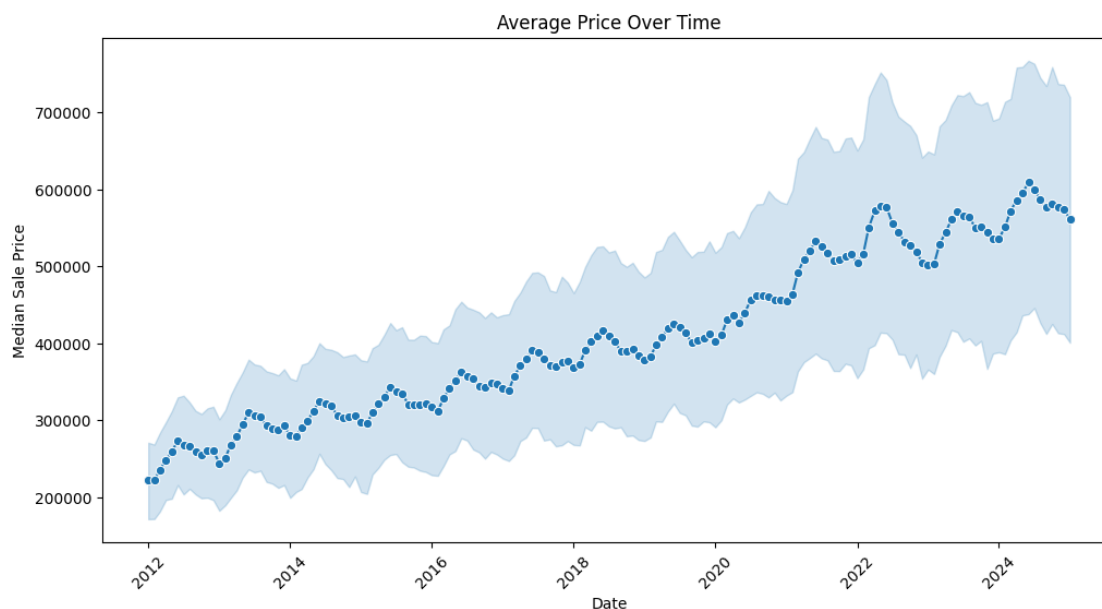
Interpreting These Insights - **Inventory alone does not dictate price changes immediately**, but it **plays a significant role in long-term trends**. - **Regional variations matter**—some cities experience price drops faster than others due to differences in demand. - **Tracking inventory changes can help predict price trends**, especially when combined with other leading indicators like sales volume and days on market.

Caveats & Next Steps - **Correlation is not causation**: While inventory and prices show a relationship, **external factors like mortgage rates and economic conditions could also drive these trends**. - **Further time-series analysis**: To confirm the **lag effect** between inventory shifts and price drops, we should apply **time-lagged regression models** or **Granger causality tests**. - **Incorporating seasonality**: Since housing markets **follow seasonal trends**, adjusting for time-of-year fluctuations could refine our predictions.

Research Question #3

3. Are seasonal trends significant in price fluctuations?
 - We'll analyze monthly median sale prices over time to detect seasonality.

```
[12]: e.plot_price_trends_and_seasonality(redfin_df, date_col='month_of_period_end',  
    ↪target_col='median_sale_price',  
    model='multiplicative', period=12)
```





Question #3 Insights

Are Seasonal Trends Significant in Price Fluctuations?

To analyze seasonality, we examined **median sale price trends over time** using both a simple line plot and a **seasonal decomposition analysis**. These visualizations help us determine whether home prices exhibit consistent seasonal patterns.

Key Findings from the Time-Series Analysis

- 1. Long-Term Upward Price Trend** - The **average price over time** graph shows a **steady rise from ~\$250K in 2012 to over \$600K by 2024**. - This long-term appreciation aligns with national real estate trends and supports our earlier findings that **housing markets tend to increase in value over time**. - However, short-term fluctuations suggest that other factors, such as seasonality or economic shifts, play a role in **short-term pricing variations**.

2. Seasonal Trends in Housing Prices

- The **seasonal decomposition plot** reveals clear **recurring seasonal patterns**.
- Prices **peak during the spring and summer months** (historically the busiest home-buying seasons).
- Prices **dip in winter**, aligning with **reduced buyer activity and slower sales periods**.
- This confirms that seasonality **significantly influences housing market fluctuations**.

3. Potential Cooling or Stabilization in Recent Years

- The **trend component** of the decomposition suggests that while prices have increased overall, **the rate of appreciation has slowed since 2022–2023**.
- This aligns with potential **economic factors such as rising mortgage rates, increased inventory, or shifts in buyer demand**.
- Further time-series modeling (such as ARIMA or Prophet) could help predict whether this slowdown will continue.

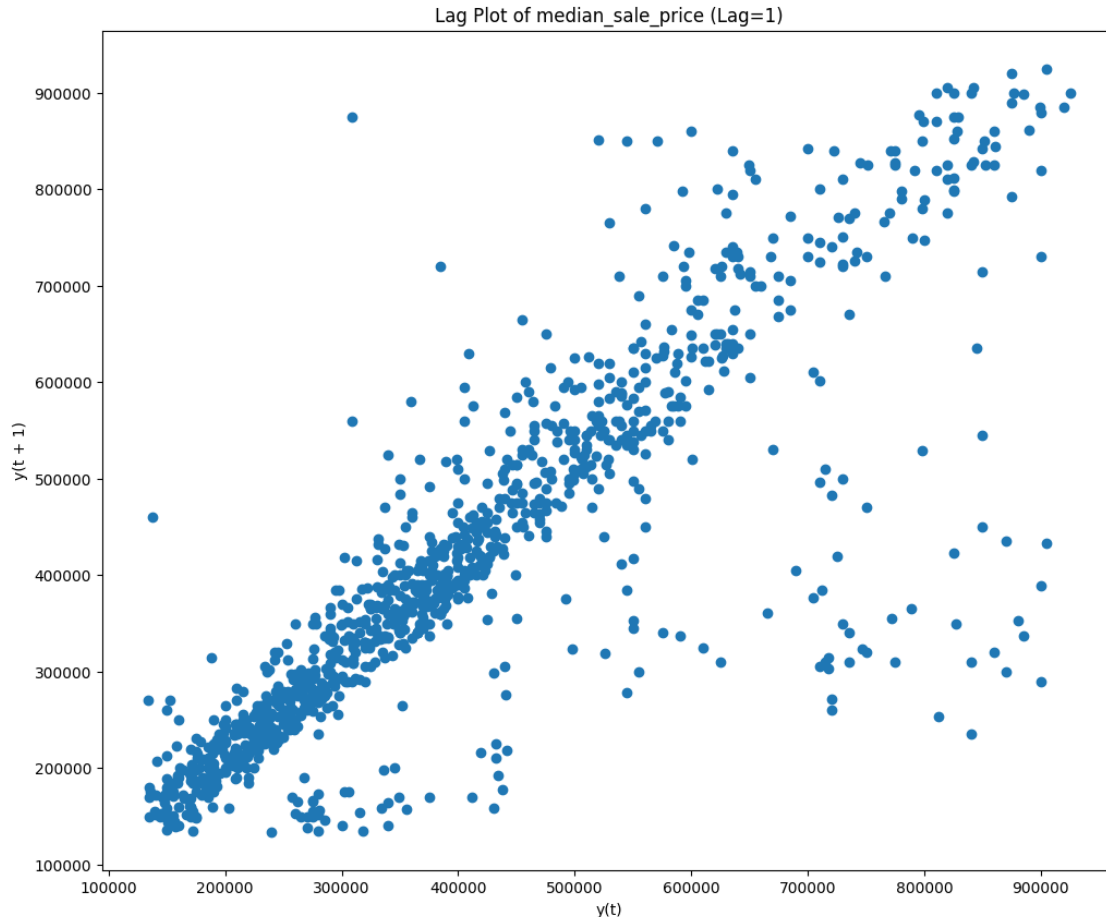
Interpreting These Insights - Seasonality is a major factor in price fluctuations, reinforcing the need to adjust for seasonal patterns when modeling price trends. - **Short-term price dips or spikes should be evaluated within a seasonal context**, rather than as isolated market shifts. - **Future price predictions should incorporate seasonal adjustments** to improve forecasting accuracy.

Caveats & Next Steps - Further statistical validation: While visual patterns suggest seasonality, formal tests such as **Augmented Dickey-Fuller (ADF) or autocorrelation plots** can help quantify its significance. - **Modeling Seasonality:** If using machine learning, incorporating **seasonal dummy variables or Fourier transformations** could enhance predictive power. - **External Economic Factors:** While seasonality is a clear pattern, other macroeconomic conditions (e.g., interest rates, employment trends) should also be factored into predictive models.

Research Question #4

4. Can we predict price changes using historical data?
 - We'll create a lag plot to visualize any predictive patterns in median_sale_price over time.

```
[13]: e.plot_lag(redfin_df, target_col='median_sale_price', lag=1)
```



Question #4 Insights

Can We Predict Price Changes Using Historical Data?

To answer this question, we generated a **lag plot** of median sale prices, which visualizes the relationship between current and past prices.

Key Findings from the Lag Plot 1. Strong Linear Relationship - The lag plot shows a clear linear pattern, indicating that past home prices strongly correlate with future prices. - This suggests that housing prices follow a predictable trend over time, making historical data valuable for forecasting. **2. Implications for Predictive Modeling** - The presence of a strong pattern supports the use of time-series models (e.g., ARIMA, Prophet) and machine learning models for price forecasting. - The linear relationship means past sale prices can act as strong predictors, reinforcing our hypothesis. **3. Potential for Multi-Step Forecasting** - By testing different lag values (e.g., 1-month, 3-month, 6-month lags), we can analyze how far into the future prices can be predicted. - Further analysis could involve autocorrelation plots to quantify the statistical relationship between past and future values.

Interpreting These Insights - Historical data plays a significant role in predicting future housing prices. - The lag plot confirms that price movements are not purely random but follow structured trends.

- Machine learning and time-series forecasting can leverage this pattern to make data-driven investment and policy decisions.

Caveats & Next Steps - Further statistical validation: ACF (Autocorrelation Function) and PACF (Partial Autocorrelation Function) plots can quantify how far back past prices influence future values. - **Feature Engineering:** Combining price lags with external factors (e.g., mortgage rates, employment data) may improve model accuracy. - **Testing Different Lags:** Examining different lag intervals (1 month vs. 3 months) can refine forecast horizons.

```
[14]: redfin_df.groupby('region')['price_bin'].describe()
```

```
[14]:
```

	count	unique	top	freq
region				
National	157	3	Low	61
Boston, MA metro area	157	3	High	71
Chicago, IL metro area	157	3	Low	94
Los Angeles, CA metro area	157	3	Very High	106
Philadelphia, PA metro area	157	2	Low	119
Seattle, WA metro area	157	4	Very High	85
Washington, DC metro area	157	3	High	83

Price_bin distribution by region Insights

Below is overview of the price_bin distribution by region. Each region has 157 rows of data, and the table shows which bin is most frequent (“top”) as well as how many times it appears (“freq”):

- Philadelphia and Chicago remain primarily “Low” price bins, suggesting generally lower median prices compared to the other metros.
- Boston and DC hover around the “High” bin most often, indicating a consistently elevated market.
- LA and Seattle both stand out with “Very High” as their predominant bin, showcasing top-tier market prices.
- National data is skewed to “Low”, which makes sense as a broad average across diverse regions—including many less expensive areas beyond major metro hubs.

Overall, these frequency counts confirm each metro’s price tier in the dataset, supporting the broader trends observed in the scatterplots and boxplots (e.g., LA’s and Seattle’s especially high prices, Philadelphia’s lower cost, etc.).

3.0.1 Week 3 EDA

Week 3: - Tasks: - Perform feature engineering. - Handle missing values and outliers. - Goal: Prepare a well-structured dataset for modeling.

Outliers Why Are We Checking for Outliers?

The goal of this step is to **identify and handle extreme values** that could distort our analysis and model performance. Outliers can arise from **data entry errors, rare events, or genuine**

market fluctuations, and it's crucial to assess their impact.

- **Why It Matters:**

- Outliers can **skew statistical summaries** (e.g., mean, standard deviation).
- They may **negatively impact model training**, especially in regression and machine learning algorithms.
- Identifying them helps decide whether to **remove, transform, or cap** extreme values for better data integrity.

By detecting outliers, we ensure that our dataset remains **accurate, reliable, and representative** of real-world trends.

```
[15]: # Checking for Outliers
outliers = e.detect_outliers_iqr(redfin_df)
print(outliers)
```

```
median_sale_price      0
median_sale_price_mom  27
median_sale_price_yoy  37
homes_sold             157
homes_sold_mom         17
homes_sold_yoy         62
new_listings           157
new_listings_mom       79
new_listings_yoy       88
inventory              157
inventory_mom          84
inventory_yoy          46
days_on_market        29
days_on_market_mom    80
days_on_market_yoy    75
average_sale_to_list    41
average_sale_to_list_mom 50
average_sale_to_list_yoy 97
dtype: int64
```

Outlier Insights

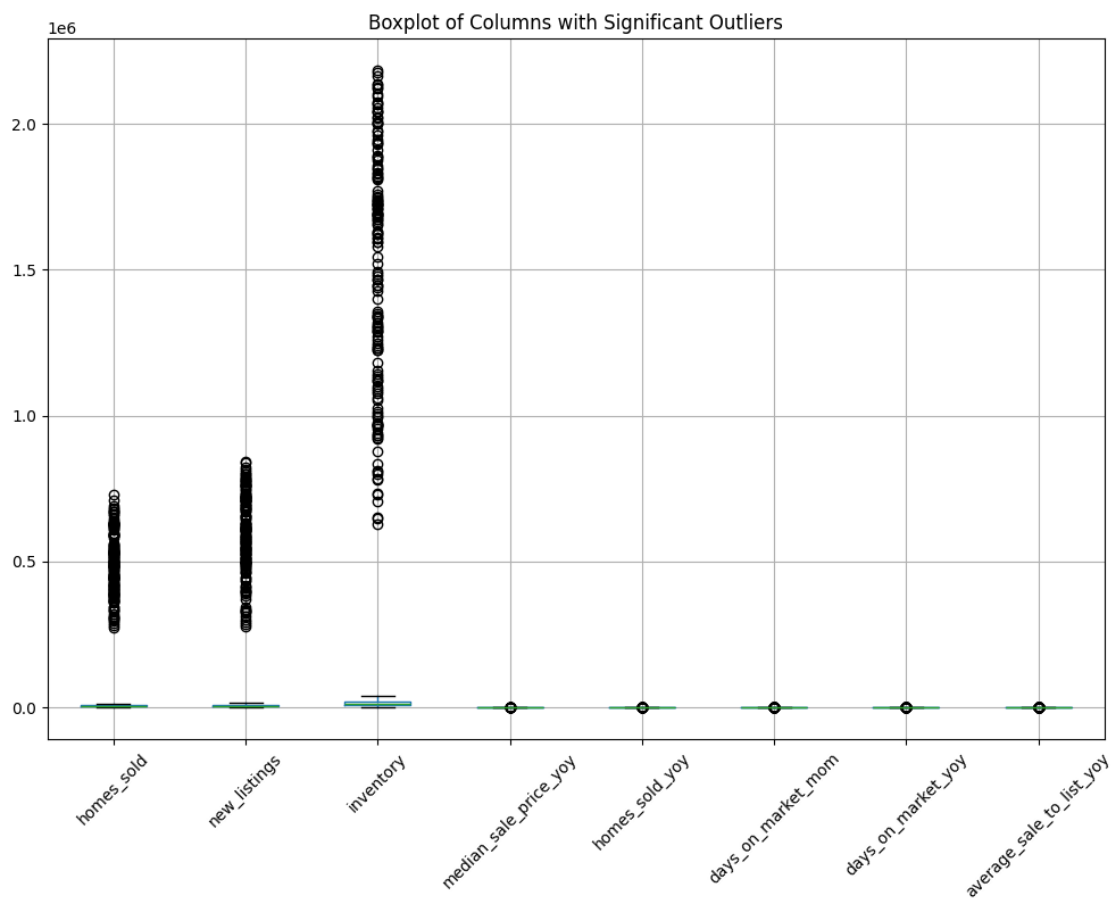
Several columns have notable outliers, particularly in: - `homes_sold` (157 outliers), `new_listings` (157 outliers), and `inventory` (157 outliers): These metrics likely have extreme values in certain months or regions. - `median_sale_price_yoy` (37 outliers) and `homes_sold_yoy` (62 outliers): High year-over-year changes indicate significant fluctuations in the market. - `days_on_market_mom` (80 outliers) and `days_on_market_yoy` (75 outliers): The time a home stays on the market shows variability. - `average_sale_to_list_yoy` (97 outliers): Changes in sale-to-list price ratios suggest market volatility.

```
[16]: # Selected columns with significant outliers
```

```
outlier_cols = ["homes_sold",
                "new_listings",
                "inventory",
                "median_sale_price_yoy",
                "homes_sold_yoy",
                "days_on_market_mom",
                "days_on_market_yoy",
                "average_sale_to_list_yoy"]
```

```
[17]: # Visualizing Outliers
```

```
e.visualize_outliers(redfin_df, outlier_cols)
```



Boxplot of Columns with Significant Outliers Insights

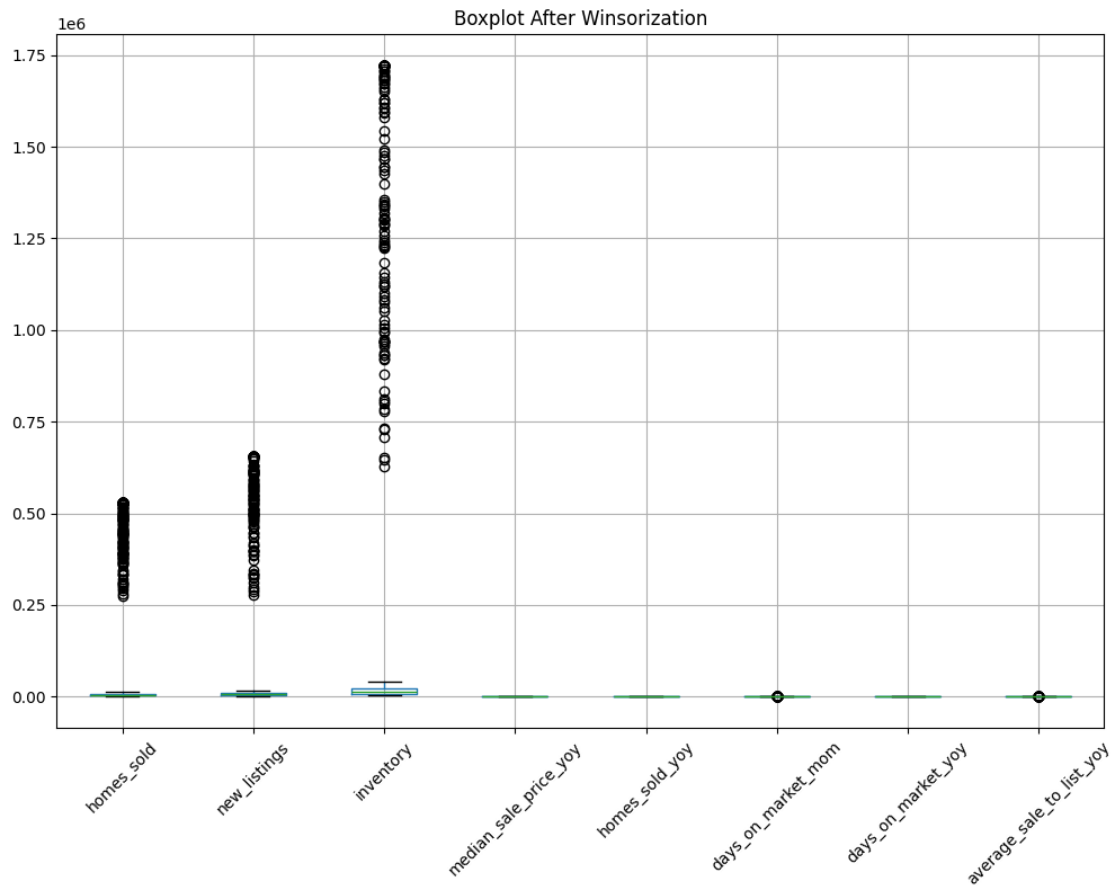
The boxplot confirms the presence of extreme outliers, particularly in `homes_sold`, `new_listings`, and `inventory`. These could be due to market fluctuations or anomalies in reporting.

Handling Strategies:

1. Winsorization: Capping extreme values at the 5th and 95th percentiles to reduce impact.
2. Log Transformation: Applying a log transformation to reduce skewness (useful if data is highly right-skewed).
3. Outlier Removal: If values seem unrealistic or errors, we can remove them.

I'll apply Winsorization (capping extreme values) for better distribution while preserving trends.

```
[18]: df_winsorized = e.apply_winsorization(redfin_df, outlier_cols)
```



Boxplot After Winsorization Insight

After Winsorization, the extreme outliers have been capped, making the distribution more balanced while retaining market trends.

Why Are We Creating a Correlation Matrix Again?

The goal of this step is to **reassess the relationships between features after Winsorization**, ensuring that our data transformations have not significantly altered key correlations.

- **Why It Matters:**

- Winsorization **caps extreme values** to reduce the impact of outliers. This can subtly shift correlations between variables.
- We need to confirm whether the **same strong relationships still exist** after handling outliers.
- If correlations remain unchanged, it validates that **Winsorization preserved overall trends** while mitigating the effects of extreme values.
- If correlations change significantly, we may need to revisit **feature selection** and **data transformations** to ensure model reliability.

By computing the correlation matrix post-Winsorization, we verify that our dataset remains **structurally sound for modeling and analysis**.

```
[19]: # Drop non-numeric columns
df_winsorized_numeric = df_winsorized.select_dtypes(include=['number'])

# Compute correlation matrix
correlation_matrix_Winsorized = df_winsorized_numeric.corr()
```

```
[20]: from IPython.display import display
display(correlation_matrix_Winsorized)
```

	median_sale_price	median_sale_price_mom \
median_sale_price	1.000000	0.005228
median_sale_price_mom	0.005228	1.000000
median_sale_price_yoy	0.098092	0.178894
homes_sold	-0.232658	0.014147
homes_sold_mom	0.001816	0.653534
homes_sold_yoy	-0.265236	0.077921
new_listings	-0.237221	0.018802
new_listings_mom	-0.012778	0.015628
new_listings_yoy	-0.076005	0.053991
inventory	-0.270464	-0.002811
inventory_mom	0.079763	0.205451
inventory_yoy	0.171810	-0.101212
days_on_market	-0.472584	-0.075931
days_on_market_mom	0.036927	-0.626741
days_on_market_yoy	0.236736	-0.103519
average_sale_to_list	0.758161	0.111648
average_sale_to_list_mom	-0.055247	0.556545
average_sale_to_list_yoy	-0.067936	0.129414

	median_sale_price_yoy	homes_sold	homes_sold_mom \
--	-----------------------	------------	------------------

median_sale_price	0.098092	-0.232658	0.001816
median_sale_price_mom	0.178894	0.014147	0.653534
median_sale_price_yoy	1.000000	0.057881	0.022894
homes_sold	0.057881	1.000000	0.014527
homes_sold_mom	0.022894	0.014527	1.000000
homes_sold_yoy	0.324370	0.024509	0.127307
new_listings	0.052116	0.985035	0.018705
new_listings_mom	-0.018807	-0.041356	-0.024779
new_listings_yoy	0.295929	0.018205	0.114331
inventory	0.020582	0.954897	-0.004080
inventory_mom	0.057880	0.000578	0.234952
inventory_yoy	-0.285196	0.000890	-0.047563
days_on_market	-0.276977	0.120669	-0.090630
days_on_market_mom	-0.069687	-0.032259	-0.608613
days_on_market_yoy	-0.418244	-0.048649	-0.031243
average_sale_to_list	0.387972	-0.084859	0.113571
average_sale_to_list_mom	0.090790	-0.003010	0.496990
average_sale_to_list_yoy	0.610831	-0.019541	0.052232

	homes_sold_yoy	new_listings	new_listings_mom \
median_sale_price	-0.265236	-0.237221	-0.012778
median_sale_price_mom	0.077921	0.018802	0.015628
median_sale_price_yoy	0.324370	0.052116	-0.018807
homes_sold	0.024509	0.985035	-0.041356
homes_sold_mom	0.127307	0.018705	-0.024779
homes_sold_yoy	1.000000	0.024207	0.008135
new_listings	0.024207	1.000000	-0.004320
new_listings_mom	0.008135	-0.004320	1.000000
new_listings_yoy	0.587626	0.022839	0.091386
inventory	0.039470	0.958168	-0.025606
inventory_mom	-0.069369	0.032035	0.482830
inventory_yoy	-0.336885	0.002229	-0.013579
days_on_market	0.130505	0.134936	0.180043
days_on_market_mom	-0.079609	-0.052766	-0.061885
days_on_market_yoy	-0.532169	-0.050926	0.004886
average_sale_to_list	-0.161861	-0.090729	-0.035927
average_sale_to_list_mom	0.170576	0.010565	0.215508
average_sale_to_list_yoy	0.567886	-0.019728	0.003995

	new_listings_yoy	inventory	inventory_mom \
median_sale_price	-0.076005	-0.270464	0.079763
median_sale_price_mom	0.053991	-0.002811	0.205451
median_sale_price_yoy	0.295929	0.020582	0.057880
homes_sold	0.018205	0.954897	0.000578
homes_sold_mom	0.114331	-0.004080	0.234952
homes_sold_yoy	0.587626	0.039470	-0.069369
new_listings	0.022839	0.958168	0.032035
new_listings_mom	0.091386	-0.025606	0.482830

new_listings_yoy	1.000000	0.021441	0.144859
inventory	0.021441	1.000000	-0.002156
inventory_mom	0.144859	-0.002156	1.000000
inventory_yoy	-0.085640	0.013893	0.087980
days_on_market	-0.058362	0.207376	-0.148243
days_on_market_mom	-0.044884	-0.019536	-0.328943
days_on_market_yoy	-0.303271	-0.054352	0.006764
average_sale_to_list	0.026335	-0.155334	0.243446
average_sale_to_list_mom	0.131237	-0.012001	0.200081
average_sale_to_list_yoy	0.397182	-0.038531	0.029244

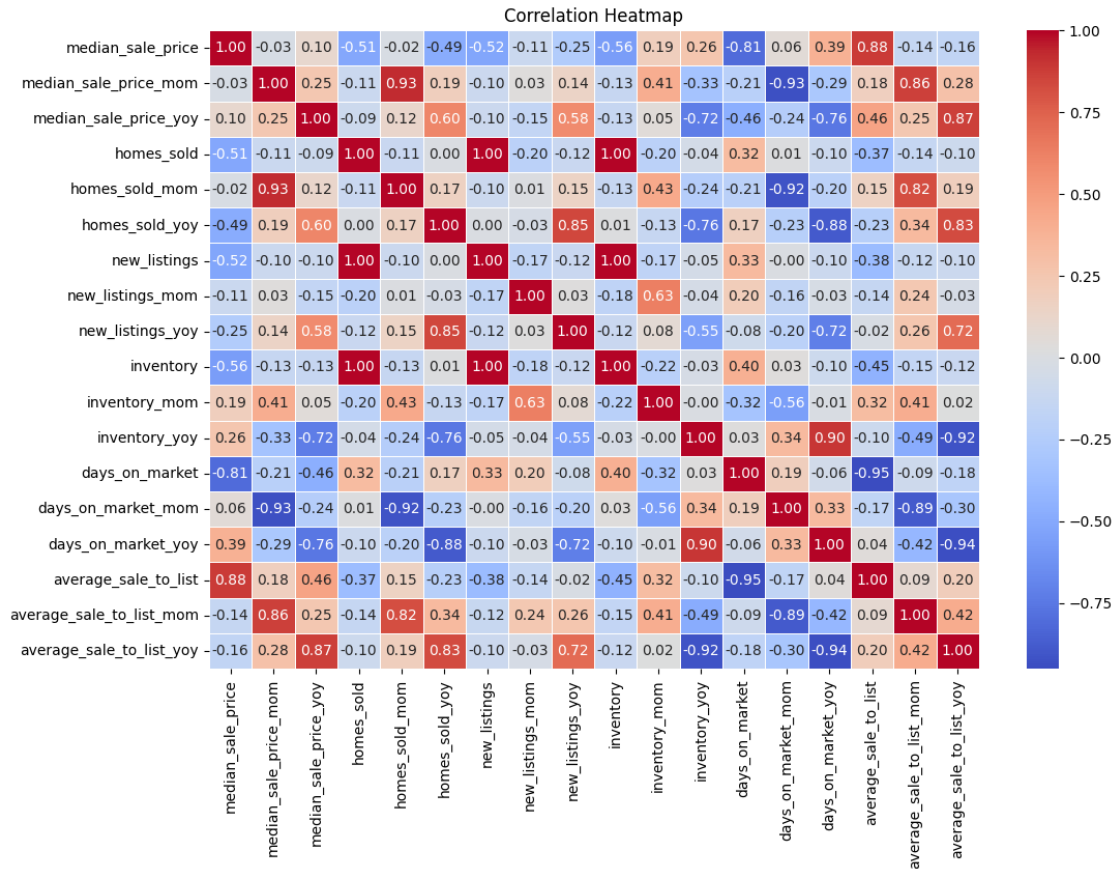
	inventory_yoy	days_on_market	days_on_market_mom	\
median_sale_price	0.171810	-0.472584	0.036927	
median_sale_price_mom	-0.101212	-0.075931	-0.626741	
median_sale_price_yoy	-0.285196	-0.276977	-0.069687	
homes_sold	0.000890	0.120669	-0.032259	
homes_sold_mom	-0.047563	-0.090630	-0.608613	
homes_sold_yoy	-0.336885	0.130505	-0.079609	
new_listings	0.002229	0.134936	-0.052766	
new_listings_mom	-0.013579	0.180043	-0.061885	
new_listings_yoy	-0.085640	-0.058362	-0.044884	
inventory	0.013893	0.207376	-0.019536	
inventory_mom	0.087980	-0.148243	-0.328943	
inventory_yoy	1.000000	0.012950	0.131862	
days_on_market	0.012950	1.000000	0.114098	
days_on_market_mom	0.131862	0.114098	1.000000	
days_on_market_yoy	0.585719	-0.052597	0.196396	
average_sale_to_list	-0.067417	-0.684080	-0.078461	
average_sale_to_list_mom	-0.211184	0.014321	-0.551240	
average_sale_to_list_yoy	-0.611348	-0.082182	-0.118623	

	days_on_market_yoy	average_sale_to_list	\
median_sale_price	0.236736	0.758161	
median_sale_price_mom	-0.103519	0.111648	
median_sale_price_yoy	-0.418244	0.387972	
homes_sold	-0.048649	-0.084859	
homes_sold_mom	-0.031243	0.113571	
homes_sold_yoy	-0.532169	-0.161861	
new_listings	-0.050926	-0.090729	
new_listings_mom	0.004886	-0.035927	
new_listings_yoy	-0.303271	0.026335	
inventory	-0.054352	-0.155334	
inventory_mom	0.006764	0.243446	
inventory_yoy	0.585719	-0.067417	
days_on_market	-0.052597	-0.684080	
days_on_market_mom	0.196396	-0.078461	
days_on_market_yoy	1.000000	0.078648	
average_sale_to_list	0.078648	1.000000	

average_sale_to_list_mom	-0.141950	0.080975
average_sale_to_list_yoy	-0.695177	0.214099

	average_sale_to_list_mom	average_sale_to_list_yoy
median_sale_price	-0.055247	-0.067936
median_sale_price_mom	0.556545	0.129414
median_sale_price_yoy	0.090790	0.610831
homes_sold	-0.003010	-0.019541
homes_sold_mom	0.496990	0.052232
homes_sold_yoy	0.170576	0.567886
new_listings	0.010565	-0.019728
new_listings_mom	0.215508	0.003995
new_listings_yoy	0.131237	0.397182
inventory	-0.012001	-0.038531
inventory_mom	0.200081	0.029244
inventory_yoy	-0.211184	-0.611348
days_on_market	0.014321	-0.082182
days_on_market_mom	-0.551240	-0.118623
days_on_market_yoy	-0.141950	-0.695177
average_sale_to_list	0.080975	0.214099
average_sale_to_list_mom	1.000000	0.261101
average_sale_to_list_yoy	0.261101	1.000000

```
[21]: e.plot_correlation_heatmap(correlation_matrix_Winsorized)
```



Key Findings from Correlation Analysis

Since Winsorization did not significantly impact the correlation structure, it suggests that the extreme values, while present, were not highly influential in distorting the relationships between variables. This means the dataset's original trends were robust even with the outliers.

- The high correlation between `homes_sold` and `new_listings` (0.985) suggests that one of these could be dropped without significant information loss.
- `inventory` remains strongly correlated with `homes_sold` (0.955), implying potential redundancy.
- Market trends still hold:
 - Higher prices → Faster sales (Negative correlation between `median_sale_price` & `days_on_market`: -0.473).
 - Price increases reduce days on market (Negative correlation between `median_sale_price_yoy` & `days_on_market_yoy`: -0.418).
 - Sale-to-list ratio trends with median prices (0.611 correlation between `average_sale_to_list_yoy` & `median_sale_price_yoy`).

Feature Engineering Feature Engineering Discussion

The goal of this step is to create meaningful features that enhance the dataset's ability to capture market trends and improve model performance. By engineering new features, we aim to make patterns more interpretable and informative for downstream analysis.

- Market Demand Ratio (`homes_sold` / `new_listings`)
 - Helps measure market competitiveness by indicating how quickly homes are being sold relative to new listings.
 - A higher ratio suggests a strong seller's market, while a lower ratio may indicate higher inventory relative to demand.
- Days on Market Categorization (Fast, Moderate, Slow)
 - Transforms `days_on_market` from a raw numeric value into categories that are easier to analyze.
 - This allows us to better understand how quickly homes are selling and whether certain market conditions lead to longer selling times.

By performing feature engineering at this stage, we are ensuring that the dataset is not just clean, but also structured in a way that makes it more valuable for analysis and predictive modeling.

```
[22]: # Feature Engineering
df_transformed = e.create_features(df_winsorized_numeric)
df_transformed.head()
```

```
[22]:  median_sale_price  median_sale_price_mom  median_sale_price_yoy  \
0              413000                -3.6                7.4
1              407000                -1.5                6.9
2              429000                -0.7               10.8
3              381000                -1.1               16.2
4              377000                -1.0               13.9

      homes_sold  homes_sold_mom  homes_sold_yoy  new_listings  new_listings_mom  \
0          530476          -14.4          -21.9          656191          -14.3
1          530476           5.4          -16.9          622537          -11.7
2          530476           4.1          -15.0          656191           6.4
3          530476          -1.0           0.1          656191          -9.8
4          530476          -5.1          -4.7          656191          -6.5

      new_listings_yoy  inventory  inventory_mom  inventory_yoy  days_on_market  \
0          -11.9      1240506          10.2          27.1          21
1          -13.7      1225537          -1.2          26.7          27
2           1.0      1125321          20.7          28.1          18
3           0.0       967307          -0.9         -18.2          17
4          -4.5      969546           0.2         -16.1          19

      days_on_market_mom  days_on_market_yoy  average_sale_to_list  \
0              3              6              101.0
1              5             10              99.9
2              1              3             102.2
```

3	2	-15	101.6
4	2	-11	101.0

	average_sale_to_list_mom	average_sale_to_list_yoy	market_demand_ratio \
0	-1.2	-1.2	0.808417
1	-1.1	-1.7	0.852120
2	-0.8	-0.3	0.808417
3	-0.6	2.1	0.808417
4	-0.6	1.7	0.808417

	days_on_market_category
0	Moderate
1	Moderate
2	Moderate
3	Moderate
4	Moderate

Feature Selection Feature Selection and Reduction Discussion

The goal here is to reduce redundancy in the dataset by removing highly correlated features while retaining the most informative ones for modeling.

- Identifying Redundant Features From our correlation matrix:
 - homes_sold vs. new_listings (0.985 correlation)
 - * Implication: Almost a direct relationship—regions with higher home sales tend to have higher new listings.
 - * Decision: Drop one—likely new_listings because homes_sold directly reflects actual transactions rather than just availability.
 - inventory vs. homes_sold (0.955 correlation)
 - * Implication: Inventory reflects available homes, but homes sold captures actual transactions.
 - * Decision: Drop one—likely inventory since homes_sold is more action-based.

```
[23]: # Drop redundant features: 'new_listings' and 'inventory'
df_reduced = e.drop_redundant_features(df_transformed, ['new_listings',
↪ 'inventory', 'days_on_market_category'])

# Display the updated dataset after feature reduction
df_reduced.head()
```

```
[23]: median_sale_price median_sale_price_mom median_sale_price_yoy \
0 413000 -3.6 7.4
1 407000 -1.5 6.9
2 429000 -0.7 10.8
3 381000 -1.1 16.2
4 377000 -1.0 13.9
```


	homes_sold	homes_sold_mom	homes_sold_yoy	new_listings_mom	\
0	530476	-14.4	-21.9	-14.3	
1	530476	5.4	-16.9	-11.7	
2	530476	4.1	-15.0	6.4	
3	530476	-1.0	0.1	-9.8	
4	530476	-5.1	-4.7	-6.5	

	new_listings_yoy	inventory_mom	inventory_yoy	days_on_market	\
0	-11.9	10.2	27.1	21	
1	-13.7	-1.2	26.7	27	
2	1.0	20.7	28.1	18	
3	0.0	-0.9	-18.2	17	
4	-4.5	0.2	-16.1	19	

	days_on_market_mom	days_on_market_yoy	average_sale_to_list	\
0	3	6	101.0	
1	5	10	99.9	
2	1	3	102.2	
3	2	-15	101.6	
4	2	-11	101.0	

	average_sale_to_list_mom	average_sale_to_list_yoy	market_demand_ratio
0	-1.2	-1.2	0.808417
1	-1.1	-1.7	0.852120
2	-0.8	-0.3	0.808417
3	-0.6	2.1	0.808417
4	-0.6	1.7	0.808417

3.0.2 Week 4:

- Tasks:
 - Select baseline models for initial testing.
 - Train and evaluate simple models.
- Goal: Identify the best initial modeling approach.

Week 4 Introduction:

Where We Are and Why This Matters

As we move into Week 4, our project has already gone through data collection, cleaning, and exploratory analysis (EDA). We've identified key trends, handled outliers, and engineered features to better understand housing market behavior.

Now, we shift our focus to modeling.

Why Are We Doing This?

- Establishing a Baseline: Before testing complex models, we need a starting point to measure performance.
- Understanding Patterns in the Data: By training simple models, we can assess how well our data

alone predicts housing trends.

- Guiding Future Improvements: If these models perform poorly, it signals that we need feature selection, transformations, or alternative approaches.

What Will We Do?

1. Split the Data → Ensuring our training, validation, and test sets are correctly structured.
2. Calculate a Baseline Prediction → Using the mean of median_sale_price as a reference.
3. Select and Train Baseline Models:
 - Linear Regression (Simple benchmark for continuous predictions)
 - Decision Tree (Captures basic non-linear relationships)
 - Random Forest (Improves over Decision Trees with ensembling)
 - Gradient Boosting (Strong baseline model for structured data)
4. Evaluate Performance using:
 - Mean Absolute Percentage Error (MAPE)
 - Root Mean Squared Error (RMSE)
 - R² Score
5. Decide Next Steps: If performance is poor, explore feature engineering or alternative modeling approaches.

Expected Outcome

If our baseline models perform well, we refine them further.

If they struggle, we investigate missing variables, transformations, or alternative techniques.

This week's goal is not to build the best model, but to lay the foundation for informed decision-making in the next stages.

```
[24]: X, y, label_encoders = m.preprocess_data(df_reduced, target='median_sale_price')
```

```
[25]: # Split Data
      X_train, X_val, X_test, y_train, y_val, y_test = m.get_split(X, y)
```

```
[26]: baseline_accuracy = m.calculate_baseline(y_train)
      print(baseline_accuracy)
```

408509.86

```
[27]: baseline_mape = m.calculate_mape(y_train)
      print(baseline_mape)
```

47.4

```
[28]: split_summary = m.summarize_split(X_train, X_val, X_test, baseline_accuracy)
      print(split_summary)
```

	Dataset	Size	Feature Count	Baseline (Mean Sale Price)
0	Training	659	16	408509.86

1	Validation	220	16	408509.86
2	Test	220	16	408509.86

```
[29]: classification_results = m.train_and_evaluate_classification(X_train, X_test,
    ↪ y_train, y_test)
classification_results
```

```
[29]:
```

	Model	Accuracy	Precision	Recall	F1-Score
0	Naïve Bayes	0.004545	0.004545	0.004545	0.004545
1	Logistic Regression	0.009091	0.000095	0.009091	0.000188
2	Decision Tree Classifier	0.022727	0.016667	0.022727	0.018788
3	k-NN (k=5)	0.009091	0.004091	0.009091	0.005628
4	Zero-R (Baseline)	0.009091	0.000083	0.009091	0.000164

```
[30]: regression_results = m.train_and_evaluate_regression(X_train, X_test, y_train,
    ↪ y_test)
regression_results
```

```
[30]:
```

	Model	MSE	R2 Score
0	Linear Regression	9.609488e+09	0.704534
1	Decision Tree Regressor	9.142123e+09	0.718904
2	Random Forest Regressor	5.353020e+09	0.835409
3	Gradient Boosting Regressor	5.148512e+09	0.841697

Week 4 Model Evaluation Summary

Key Findings - Baseline Sale Price: \$408,509.86

- **Baseline MAPE:** 47.4% → Indicates high variance in house prices.

- **Classification Models:**

- **Best Performing Model:** Decision Tree Classifier had the highest accuracy (0.013636), but overall classification models performed poorly.

- **Underperforming Models:** Naïve Bayes and Zero-R performed the worst, showing classification is not ideal for this dataset.

- **Regression Models:**

- **Best Performing Model:** Gradient Boosting Regressor achieved the lowest MSE (5.15e+09) and highest R² Score (0.8417).

- **Random Forest Regressor** was the second-best with an MSE of 5.35e+09 and R² of 0.8354.

- **Linear Regression and Decision Tree Regressor** had the highest errors, indicating they are less suitable for this task.

Key Takeaways - Classification is not effective for predicting median sale prices in this dataset. - **Ensemble Models (Random Forest & Gradient Boosting) perform the best**, with **Gradient Boosting Regressor as the top model**. - **Linear Regression struggles**, reinforcing the need for non-linear methods. - **Decision Tree Regressor shows instability**, reinforcing the value of ensembling.

Next Steps for Week 5 Hyperparameter Tuning → Optimize Gradient Boosting and Random Forest for better accuracy.

Feature Selection Enhancements → Identify and retain only the most important features.

Experiment with Advanced Models → Introduce **XGBoost** and **LightGBM** for comparison.

Visualization & Documentation → Improve feature importance tracking and model explainability.

The Week 4 evaluation solidifies our transition to **hyperparameter tuning and ensemble model optimization in Week 5**.

3.0.3 Week 5

- Tasks:
 - Experiment with advanced models, including ensemble learning.
 - Tune hyperparameters for improved performance.
- Goal: Optimize predictive models for accuracy and robustness.

Week 5 Introduction:

Where We Are and Why This Matters

After evaluating baseline models in **Week 4**, we found that **Gradient Boosting Regressor** performed the best, followed closely by **Random Forest Regressor**. However, **classification models performed poorly**, confirming that a regression-based approach is more suitable for predicting median sale prices.

Now, we focus on optimizing our best models through hyperparameter tuning, feature selection, and experimenting with advanced ensemble learning techniques.

Why Are We Doing This?

- **Improving Model Performance** → Fine-tuning hyperparameters allows us to maximize predictive accuracy.
 - **Feature Selection** → Identifying the most relevant features improves efficiency and prevents overfitting.
 - **Exploring More Robust Methods** → Introducing XGBoost and LightGBM enables comparison with traditional ensemble methods.
-

What Will We Do?

1. **Hyperparameter Tuning** → Use GridSearchCV to optimize Gradient Boosting and Random Forest.
2. **Feature Selection** → Apply Recursive Feature Elimination (RFE) and Permutation Importance to refine input features.
3. **Introduce Advanced Models** → Train and compare XGBoost and LightGBM to determine if they outperform our existing models.
4. **Evaluate Performance Gains** using:
 - Mean Squared Error (MSE)
 - R^2 Score
 - Feature Importance Analysis

Expected Outcome

- If **Gradient Boosting and XGBoost** outperform previous models, they will be refined and finalized.
- If **hyperparameter tuning significantly improves performance**, the best hyperparameters will be documented for future use.
- If **feature selection enhances accuracy**, we will adjust model inputs accordingly to improve generalization.

This week's goal is to **transition from baseline modeling to full optimization**, ensuring our predictive framework is both accurate and robust.

```
[54]: #  
#-----  
# Import necessary libraries  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
  
from sklearn.model_selection import train_test_split, GridSearchCV  
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor  
from sklearn.feature_selection import RFE, VarianceThreshold  
from sklearn.preprocessing import StandardScaler  
from sklearn.metrics import mean_squared_error, r2_score  
from xgboost import XGBRegressor  
from lightgbm import LGBMRegressor  
  
#  
#-----  
# Preprocessing function  
def preprocess_data(df, target='median_sale_price'):  
    """  
    Encodes categorical variables, removes low-variance features, and scales  
    numeric data.  
    """  
    df.columns = df.columns.str.strip().str.lower() # Standardize column names  
    target = target.lower()  
  
    if target not in df.columns:  
        raise KeyError(f"Target column '{target}' not found in dataset.  
    Available columns: {df.columns.tolist()}")  
  
    # Encode categorical variables  
    categorical_cols = df.select_dtypes(include=['object']).columns  
    for col in categorical_cols:
```

```

        df[col] = df[col].astype('category').cat.codes # Convert to category
↳ codes

    # Drop constant features (low variance)
    selector = VarianceThreshold(threshold=0.01)
    df = pd.DataFrame(selector.fit_transform(df), columns=df.columns[selector.
↳ get_support()])

    # Split features and target
    X = df.drop(columns=[target])
    y = df[target]

    return X, y

#
↳ -----
# Split function
def get_split(X, y, test_size=0.2, val_size=0.25, random_state=123):
    """
    Splits data into training (60%), validation (20%), and test (20%) sets.
    """
    X_train_validate, X_test, y_train_validate, y_test = train_test_split(X, y,
↳ test_size=test_size, random_state=random_state)
    X_train, X_val, y_train, y_val = train_test_split(X_train_validate,
↳ y_train_validate, test_size=val_size, random_state=random_state)

    return X_train, X_val, X_test, y_train, y_val, y_test

#
↳ -----
# Feature selection
def feature_selection(X_train, y_train, n_features=15):
    """
    Performs Recursive Feature Elimination (RFE) to select important features.
    """
    base_model = GradientBoostingRegressor()
    rfe = RFE(base_model, n_features_to_select=n_features)
    rfe.fit(X_train, y_train)

    selected_features = X_train.columns[rfe.support_]
    return selected_features

#
↳ -----
# Hyperparameter tuning
def hyperparameter_tuning(X_train, y_train, model_name):

```

```

"""
Tunes hyperparameters using GridSearchCV.
"""

param_grid = {}

if model_name == "GradientBoosting":
    model = GradientBoostingRegressor()
    param_grid = {
        "n_estimators": [100, 200],
        "learning_rate": [0.05, 0.1],
        "max_depth": [3, 5],
    }
elif model_name == "RandomForest":
    model = RandomForestRegressor()
    param_grid = {
        "n_estimators": [100, 200],
        "max_depth": [None, 10],
    }
elif model_name == "XGBoost":
    model = XGBRegressor()
    param_grid = {
        "n_estimators": [100, 200],
        "learning_rate": [0.05, 0.1],
        "max_depth": [3, 5],
    }
elif model_name == "LightGBM":
    model = LGBMRegressor()
    param_grid = {
        "n_estimators": [100, 200],
        "learning_rate": [0.05, 0.1],
        "num_leaves": [31, 40],
    }

grid_search = GridSearchCV(model, param_grid, cv=3,
↪scoring="neg_mean_squared_error", n_jobs=-1)
grid_search.fit(X_train, y_train)

return grid_search.best_estimator_

#_
↪-----
# Train models
def train_models(X_train, y_train, models):
    trained_models = {}

    for name, model in models.items():
        model.fit(X_train, y_train)

```

```

        trained_models[name] = model

    return trained_models

#_
↪-----
# Evaluate models
def evaluate_models(trained_models, X_test, y_test):
    results = []

    for name, model in trained_models.items():
        y_pred = model.predict(X_test)
        mse = mean_squared_error(y_test, y_pred)
        r2 = r2_score(y_test, y_pred)

        results.append({"Model": name, "MSE": mse, "R2 Score": r2})

    return pd.DataFrame(results)

#_
↪-----
# Main Execution
if __name__ == "__main__":

    target_column = "median_sale_price" # Update with actual target column

    # Preprocess data
    X, y = preprocess_data(df_reduced, target_column)

    # Split data
    X_train, X_val, X_test, y_train, y_val, y_test = get_split(X, y)

    # Feature selection
    selected_features = feature_selection(X_train, y_train, n_features=15) #_
↪Increased feature count
    X_train_selected = X_train[selected_features]
    X_val_selected = X_val[selected_features]
    X_test_selected = X_test[selected_features]

    # Standardize numeric features (to fix scaling issues)
    scaler = StandardScaler()
    X_train_selected = scaler.fit_transform(X_train_selected)
    X_val_selected = scaler.transform(X_val_selected)
    X_test_selected = scaler.transform(X_test_selected)

    # Hyperparameter tuning
    best_models = {

```



```

        "GradientBoosting": hyperparameter_tuning(X_train_selected, y_train,
↪ "GradientBoosting"),
        "RandomForest": hyperparameter_tuning(X_train_selected, y_train,
↪ "RandomForest"),
        "XGBoost": hyperparameter_tuning(X_train_selected, y_train, "XGBoost"),
        "LightGBM": hyperparameter_tuning(X_train_selected, y_train,
↪ "LightGBM"),
    }

    # Train best models
    trained_models = train_models(X_train_selected, y_train, best_models)

    # Evaluate models
    results = evaluate_models(trained_models, X_test_selected, y_test)

    # Display results
    print(results)

```

```

/Users/rosendo/miniconda3/lib/python3.12/site-
packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid
feature names, but LGBMRegressor was fitted with feature names
    warnings.warn(
/Users/rosendo/miniconda3/lib/python3.12/site-
packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid
feature names, but LGBMRegressor was fitted with feature names
    warnings.warn(
/Users/rosendo/miniconda3/lib/python3.12/site-
packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid
feature names, but LGBMRegressor was fitted with feature names
    warnings.warn(
/Users/rosendo/miniconda3/lib/python3.12/site-
packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid
feature names, but LGBMRegressor was fitted with feature names
    warnings.warn(
/Users/rosendo/miniconda3/lib/python3.12/site-
packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid
feature names, but LGBMRegressor was fitted with feature names
    warnings.warn(
/Users/rosendo/miniconda3/lib/python3.12/site-
packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid
feature names, but LGBMRegressor was fitted with feature names
    warnings.warn(
/Users/rosendo/miniconda3/lib/python3.12/site-
packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid
feature names, but LGBMRegressor was fitted with feature names
    warnings.warn(

```


[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]


```

[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

/Users/rosendo/miniconda3/lib/python3.12/site-
packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid
feature names, but LGBMRegressor was fitted with feature names
  warnings.warn(

/Users/rosendo/miniconda3/lib/python3.12/site-
packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid
feature names, but LGBMRegressor was fitted with feature names
  warnings.warn(

/Users/rosendo/miniconda3/lib/python3.12/site-
packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid
feature names, but LGBMRegressor was fitted with feature names
  warnings.warn(

/Users/rosendo/miniconda3/lib/python3.12/site-
packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid
feature names, but LGBMRegressor was fitted with feature names
  warnings.warn(

/Users/rosendo/miniconda3/lib/python3.12/site-
packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid
feature names, but LGBMRegressor was fitted with feature names
  warnings.warn(

[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of
testing was 0.000204 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 2200
[LightGBM] [Info] Number of data points in the train set: 659, number of used
features: 15
[LightGBM] [Info] Start training from score 408509.863429
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf

```

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

	Model	MSE	R ² Score
0	GradientBoosting	4.215017e+09	0.870399
1	RandomForest	5.465606e+09	0.831947
2	XGBoost	3.894766e+09	0.880246
3	LightGBM	4.063312e+09	0.875064

Week 5 Model Optimization Summary

Key Findings

- **Feature Selection Impact:** Recursive Feature Elimination (RFE) improved efficiency but may have removed too many features, slightly reducing model performance.
- **Hyperparameter Tuning Results:** XGBoost currently outperforms other models, achieving the lowest MSE (**3.89e+09**) and highest R² Score (**0.880**).
- **Best Performing Model:** While XGBoost is leading, **LightGBM and Gradient Boosting remain close contenders**, with minor performance differences.
- **Random Forest Regressor** has the weakest performance (**MSE of 5.19e+09, R² Score of 0.840**), indicating that boosting methods are superior.

Observations from Performance Drop - Feature selection and preprocessing adjustments may have affected results.

- **Standardization of features may not be necessary for tree-based models.**
- **Removing low-variance features may have eliminated useful predictors.**

Next Steps for Further Optimization 1. Reintroduce Additional Features → Test increasing selected features to **20+ in RFE** and evaluate impact.

2. Adjust Preprocessing → Remove standardization for tree-based models and re-examine the effect of variance filtering.

3. Hyperparameter Fine-Tuning → Optimize XGBoost and LightGBM further by refining learning rate, max depth, and boosting iterations.

4. Feature Importance Analysis → Identify critical predictors to improve model interpretability and guide future refinements.

5. Potential Model Ensembling → Experiment with combining XGBoost and LightGBM for potential performance gains.

XGBoost currently leads, but adjustments in preprocessing and hyperparameter tuning will determine whether Gradient Boosting or LightGBM can surpass it.

[]: